



The Open University

M263 Building Blocks of Software

Unit 13

Recursion

Study guide

Sections 1 to 3 of this unit are of about equal length and importance. Section 4 is a short reading section.

To write recursive descriptions and implementations can be quite a hard task at first, and requires some practice. Do not be surprised if initially you find some of the material in this unit difficult.

There are practical computing activities associated with all four sections. You should not spend more than about 10% of your study time for this unit on practical work.

This publication forms part of an Open University course. Details of this and other Open University courses can be obtained from the Course Information and Advice Centre, PO Box 724, The Open University, Milton Keynes, MK7 6ZS, United Kingdom: tel. +44 (0)1908 653231, e-mail general-enquiries@open.ac.uk

Alternatively, you may visit the Open University website at <http://www.open.ac.uk> where you can learn more about the wide range of courses and packs offered at all levels by the Open University.

To purchase a selection of Open University course materials, please visit the Open University webshop at www.ouw.co.uk, or contact Open University Worldwide, Michael Young Building, Walton Hall, Milton Keynes MK7 6AA, United Kingdom for a brochure. tel. +44 (0)1908 858785; fax +44 (0)1908 858787; e-mail ouwenq@open.ac.uk

The Open University, Walton Hall, Milton Keynes, MK7 6AA.

First published 2004. Second edition 2008.

Copyright © 2008 The Open University

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, transmitted or utilised in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without written permission from the publisher or a licence from the Copyright Licensing Agency Ltd. Details of such licences (for reprographic reproduction) may be obtained from the Copyright Licensing Agency Ltd of 90 Tottenham Court Road, London W1T 4LP.

Edited, designed and typeset by The Open University, using the Open University T_EX System.

Printed and bound in the United Kingdom by The Charlesworth Group, Wakefield.

ISBN 978 07492 2305 2

Contents

Introduction	4
1 Recurrence systems	6
2 Sequence-based types	16
2.1 Recursion on stacks	17
2.2 Recursion on vectors	22
2.3 Recursion on strings	24
3 Binary trees	29
3.1 Recursion on binary trees	29
3.2 Traversing binary trees	33
3.3 Searching in a binary tree	37
4 Recursion and Games	41
Summary of Unit 13	44
Solutions to the exercises	45
Index	48

Introduction

The specification of a function sets out: what inputs are required, and their type; what type of value is returned; any precondition that the inputs must satisfy; and the postcondition that the returned value must satisfy. Before we write any code, we should be clear of the specification of the function it is to implement.

In Block 4, we shall be concerned with features relating to implementation. In this unit we consider a technique, known as recursion, which can be used both in specification and implementation. In Unit 14, we will be concerned with a method that can be used to prove that an implementation correctly implements a function specification, while in Unit 15, we will consider the efficiency of an implementation.

In Unit 5, Section 4, we looked at the family of functions: $SPIKE_0(t, side)$, $SPIKE_1(t, side)$, $SPIKE_2(t, side)$, ... , $SPIKE_n(t, side)$. These gave rise to a family of turtle diagrams, where each one depends on the previous diagram, and with the simplest diagram (for $SPIKE_0$) being a straight line. We mentioned that we can combine the family of functions $SPIKE_n$ into a single function by taking the index n as an additional input. The implementation of $SPIKE(n, t, side)$ given in Unit 5 is reproduced below.

“Recursion” derives from the Latin *re-* (meaning “back” or “again”) and *currere* (meaning “to run”) so literally “to run back”.

See Unit 5, p. 44.

```
function SPIKE(n,t,side)
{ // pre: side is a multiple of 4 to the power n.
  var temp in Turtle
  temp<--t.clone()
  if (n == 0)
  {
    temp.forward(side)
  }
  else
  {
    temp<--SPIKE(n-1,temp,DIV(side,2))
    temp.store()
    temp.leftTurn()
    temp<--SPIKE(n-1,temp,DIV(side,4))
    temp.restore()
    temp<--SPIKE(n-1,temp,DIV(side,2))
  }
  return temp
}
```

This implementation is an example of **recursion**, the topic of this unit. The key feature which makes the implementation recursive is that to find $SPIKE(n, t, side)$ it calls on $SPIKE(n-1, t, side)$. When executed, the implementation will call itself repeatedly, each time with a smaller value of n . The self-reference will stop when we reach $n = 0$. The code for this case is given as part of the implementation, in the **if** clause (and just draws a single turtle line).

Figure I1 illustrates a family of turtle diagrams obtained in a similar way from a family of functions $BUMP_n$. As with $SPIKE_0$, $BUMP_0$ draws a straight line. Again, we can have a single function $BUMP$ that has the index n as an input, and can give a recursive implementation of $BUMP$.

See Klots, F. S., “Turtle Graphics and Mathematical Induction”, *Mathematics Teacher*, Nov. 1987 pp. 636–639, 654.

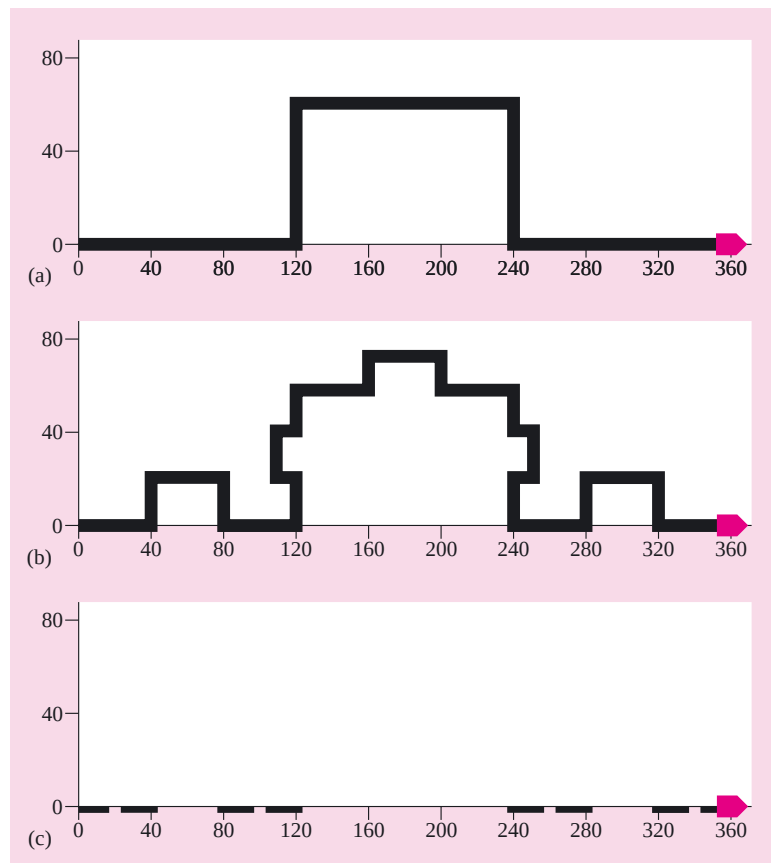


Figure 11 Turtle diagrams for (a) *BUMPy1*; (b) *BUMPy2*; (c) *BUMPy3*.

In Unit 8 we discussed binary trees, and noted that many of the functions introduced there are most naturally implemented using recursion. For example, we described in-order tree traversal as below, and observed that this description is recursive.

Form a sequence as follows. Write down the in-order traversal of the left subtree, then write down the root, then write down the in-order traversal of the right subtree. (The in-order traversal of an empty tree is the empty sequence.)

For certain implementation problems, recursion provides a very convenient form of solution. If we can solve the problem by assuming that we can solve smaller versions of the same problem, then we can use the technique. We must also provide a “stopping condition”: a simple version of the problem that is solved directly. This is to ensure that the process of “self-reference”, to smaller and smaller versions of the original problem, will eventually stop.

For example, think about how you might implement a function whose input is a binary tree of integers, and which returns the sum of all the integers in the tree. To do this without recursion would require some way of visiting every node in the tree, and would be quite tiresome. To do it recursively, we can draw on the following insight.

If the tree is empty, the sum is 0.

If the tree is not empty, the required sum is: the root + the sum of the numbers in the left subtree + the sum of the numbers in the right subtree.

In such a situation recursion can provide a solution that is easy to code, and with relatively simple code it is easier to be confident that the code is correct. As an implementation technique, however, recursion suffers from one disadvantage. It can be rather “costly” in terms of execution time,

Implementations of these functions are provided in the WorkPad files for this unit.

See Unit 8, p. 27.

Virtually all programming languages allow the use of recursive implementations.

and even more so in terms of the machine storage required. Despite this drawback, recursion remains an important topic for a number of reasons.

- It provides a convenient way of “prototyping” an implementation design, even when efficiency considerations may be an issue for the final solution.
- It provides an important tool in formally specifying what certain functions are required to do.
- The topic is deeply embedded in the theory of computation.

In Section 1 of this unit we look at recursion on integers. We introduce a way of describing a suitable function with signature $\mathbf{Int} \rightarrow \mathbf{Int}$, called a **recurrence system**, that is essentially recursive. Such recurrence systems are often encountered in applied mathematics.

In Section 2 we shall look at recursive implementations of functions that have sequence-based collection types as inputs, such as stacks, vectors and `mStrings`. Functions to process such collections — for example to reverse them, to sort them, and to search them — can be treated using recursion.

In Section 3 we look at recursion on binary trees. Section 4 is a short reading section, in which we discuss a recreational puzzle called “The towers of Hanoi”, where recursion is very helpful in finding a solution.

1 Recurrence systems

In this section we shall be concerned with functions on integers. We shall be particularly concerned with **recurrence systems**, which provide a way of describing a function on integers that uses self reference.

The functions we shall be concerned with here will input an integer, say n , and return an integer, and have a precondition such as $n > 0$ or $n \geq 0$. A simple way of describing such a function is to give a formula. Consider, for example, the function *SQUARE*, specified below.

function *SQUARE*(n in $\mathbf{Int}$) **return** in $\mathbf{Int}$

pre $n > 0$.

post The returned value is $n^2 = n \times n$.

Starting with the square of 1, we can list the first eight values returned by this function as follows: 1, 4, 9, 16, 25, 36, 49, 64.

Another function with the same domain is *FACTORIAL*, specified below.

function *FACTORIAL*(n in $\mathbf{Int}$) **return** in $\mathbf{Int}$

pre $n > 0$.

post The returned value is formed by multiplying together all the integers between 1 and n .

How might we calculate the first few values of this function, say for values of n between 1 and 6? To start, *FACTORIAL*(1) will be 1 (since in this special case there is only the single integer 1 to “multiply together”). So

$$\mathit{FACTORIAL}(1) = 1,$$

$$\mathit{FACTORIAL}(2) = 1 \times 2 = 2,$$

$$\mathit{FACTORIAL}(3) = 1 \times 2 \times 3 = 6,$$

$$\mathit{FACTORIAL}(4) = 1 \times 2 \times 3 \times 4 = 24,$$

$$\mathit{FACTORIAL}(5) = 1 \times 2 \times 3 \times 4 \times 5 = 120,$$

etc. Values of the function *FACTORIAL* are called **factorial numbers**.

Activity 1.1

Calculate $FACTORIAL(6)$. How did you find this value?

Discussion $FACTORIAL(6) = 1 \times 2 \times 3 \times 4 \times 5 \times 6 = 720$.

The easiest way to calculate this is to note that $FACTORIAL(6)$ is equal to $FACTORIAL(5) \times 6 = 120 \times 6$, which is 720.

If we are calculating factorial numbers in this way, the most efficient approach is to observe that the n th factorial number can be found by multiplying the previous factorial number by n . The only factorial number that we cannot find like this is the one we found first. So we have the following.

$$FACTORIAL(1) = 1 \quad (S)$$

$$FACTORIAL(n) = FACTORIAL(n-1) \times n \quad (n > 1) \quad (R)$$

Equations (S) and (R) give a complete and precise description of the postcondition of the function $FACTORIAL$. Taken together, (S) and (R) form a **recurrence system**.

In mathematics, $FACTORIAL(n)$ is written as $n!$ (read this as “n factorial”), but this is just a shorthand notation for the function, and does not tell you how to calculate its value. The notation

$$FACTORIAL(n) = 1 \times 2 \times 3 \times 4 \times \dots \times (n-1) \times n$$

may also be used. This indicates how the values are calculated, but although the dots may be understandable, they do not constitute a piece of notation whose meaning has been defined precisely. Equations (S) and (R) require only that we understand the idea of multiplication of numbers.

Now we would state the postcondition of $SQUARE$ formally as

post $SQUARE(n) == n \times n$.

In the same way, we can use (S) and (R) to express the specification of $FACTORIAL$ formally, as below.

function $FACTORIAL(n \text{ in } \mathbf{Int})$ **return in** $\mathbf{Int}$

pre $n > 0$.

post $FACTORIAL(1) == 1$.

$(n > 1) \Rightarrow (FACTORIAL(n) == FACTORIAL(n-1) \times n)$.

Now (R) describes $FACTORIAL(n)$ in terms of $FACTORIAL(n-1)$. This is an example of the sort of “self reference” that we discussed in the Introduction to the unit, and is an example of recursion.

We thought above about how we might calculate values of $FACTORIAL(n)$ starting with n equal to 1, and working forwards. This provides an iterative form of calculation, and would correspond to an implementation of the function of the form given in Example 1.4 below. First, we look at code that reflects the recursive view directly. In Example 1.1, the condition in statement (1) is referred to as the **stopping condition** — the corresponding **stopping value** is returned by statement (S), and statement (R) is known as the **recursive call**.

(To reflect the recurrence system directly, we have used more than one **return** statement. This is permitted in the M263 course code, although the usual course style is to avoid use of multiple **return** statements.)

By implication, this recurrence system specifies a function with input set $\mathbf{Int}$, and a precondition $n \geq 1$.

Multiplication is a type function of $\mathbf{Int}$, and we assume it to be understood.

The formal expression of postconditions was introduced in Unit 10, Subsection 2.4.

EXAMPLE 1.1

```

function FACTORIAL(n)
{    // recFact
if (n == 1) then          //(1)
{
    return 1              //(S)
}
else
{
    return FACTORIAL(n-1) * n    //(R)
}
}

```

Let us see how we can trace such code. Suppose that we want to follow the steps in the evaluation of $FACTORIAL(5)$ using the implementation *recFact* given in Example 1.1 (when the input n is 5).

To find $FACTORIAL(5)$, the code will first check that $5 == 1$ is **false** and then refer to statement (R). This gives

$$FACTORIAL(5) = FACTORIAL(4) \times 5.$$

Since $4 == 1$ is still **false**, the code will use (R) again, which will express $FACTORIAL(4)$ in terms of $FACTORIAL(3)$, as

$$FACTORIAL(4) = FACTORIAL(3) \times 4.$$

Since $3 == 1$ is **false**, (R) will be used again to give $FACTORIAL(3)$ as

$$FACTORIAL(3) = FACTORIAL(2) \times 3,$$

and then (R) is used again to give $FACTORIAL(2)$ as

$$FACTORIAL(2) = FACTORIAL(1) \times 2.$$

The stopping condition is now satisfied: $1 == 1$ is **true**, and so we use statement (S) to find

$$FACTORIAL(1) = 1.$$

At this point, the recursive part of the evaluation stops. We can set out this trace of *recFact* to calculate $FACTORIAL(5)$ as in Example 1.2.

In this evaluation, *FACTORIAL* refers to the value returned by the implementation *recFact*. Since the code reflects the recurrence system — and so the specification — so closely, it is easy here to be confident that the implementation is correct.

EXAMPLE 1.2

$$\begin{aligned}
 FACTORIAL(5) &= FACTORIAL(4) \times 5 && \text{(by R)} \\
 &= (FACTORIAL(3) \times 4) \times 5 && \text{(by R)} \\
 &= ((FACTORIAL(2) \times 3) \times 4) \times 5 && \text{(by R)} \\
 &= (((FACTORIAL(1) \times 2) \times 3) \times 4) \times 5 && \text{(by R)} \\
 &= (((1 \times 2) \times 3) \times 4) \times 5 && \text{(by S)} \\
 &= ((2 \times 3) \times 4) \times 5 \\
 &= (6 \times 4) \times 5 \\
 &= 24 \times 5 = 120.
 \end{aligned}$$

The trace in Example 1.2 illustrates a couple of points that are crucial to ensure that recursion works. Firstly, there must be a case where the recursion will stop. This is given by (S) in Example 1.1. Secondly, the recursive call must be structured so that the stopping condition will eventually hold for any valid input. The code in Example 1.3 illustrates an attempt to use recursion that is not valid.

EXAMPLE 1.3

Suppose that the following code is intended to implement a function whose input is an integer n with $n > 0$.

```
function WRONG(n)
{    // invalidRec
if (n == 1) then          //(1)
{
    return 1              //(S)
}
else
{
    return WRONG(n-2) * n  //(R)
}
}
```

Suppose we try to use this code to evaluate $WRONG(4)$. We get

$$\begin{aligned} WRONG(4) &= WRONG(2) \times 4 && \text{(by R)} \\ &= (WRONG(0) \times 2) \times 4 && \text{(by R)} \\ &= ((WRONG(-2) \times 0) \times 2) \times 4 && \text{(by R)} \\ &= \dots \end{aligned}$$

The stopping condition will never be reached so the process will not stop.

Earlier, we calculated some values of the function *FACTORIAL* by starting with *FACTORIAL*(1), then finding *FACTORIAL*(2) then *FACTORIAL*(3), and so on. This corresponds to an implementation of *FACTORIAL* that is iterative rather than recursive, as in Example 1.4.

EXAMPLE 1.4

An iterative implementation of *FACTORIAL* is given below. We start with a local variable *result* set to 1, then repeatedly multiply the number by 2, then 3, then 4, and so on.

```
function FACTORIAL(n)
{    //itFact
var i in Int
var result in Int
result <-- 1
for (i <-- 2 to n)
{
    result <-- result * i
}
return result
}
```

The recursion in this example is not very complicated, and it is because of this that we can move from a recursive view to an iterative view so easily.

We used a formula to give the postcondition of *SQUARE*. If there were such a formula for *FACTORIAL*, we could use that as the basis for a simple implementation — but no such formula exists.

Aside

Factorials are important in various branches of mathematics, such as combinatorics and probability theory. For example, there are $FACTORIAL(n)$ ways we can arrange n distinct objects in a sequence. Factorial numbers grow very quickly. For instance:

$$FACTORIAL(21) = 51\,090\,942\,171\,709\,440\,000.$$

End of Aside

We now turn to a different example. Consider a function $INTSUM$, whose input is an integer n , with $n \geq 1$, and which returns the sum of the integers between 1 and n . For example:

$$INTSUM(1) = 1,$$

$$INTSUM(2) = 1 + 2 = 3,$$

$$INTSUM(3) = 1 + 2 + 3 = 6,$$

$$INTSUM(4) = 1 + 2 + 3 + 4 = 10,$$

and so on.

To express the postcondition precisely, we can again use a recurrence system.

Activity 1.2

- Suppose that you know what $INTSUM(n - 1)$ is. How would you then find $INTSUM(n)$?
- Give a recurrence system satisfied by $INTSUM$.
- Give a formal specification of the function $INTSUM$.

Discussion

(a) We can find $INTSUM(5)$, for example, by adding 5 to $INTSUM(4)$. In general, we can find $INTSUM(n)$ by adding n to $INTSUM(n - 1)$.

(b) To find $INTSUM(1)$, we only have one number to “add up” – the integer 1. That will provide a stopping condition. We can deduce the recursive call in the recurrence system from the idea in part (a). We get the following.

$$INTSUM(1) = 1 \quad (S)$$

$$INTSUM(n) = INTSUM(n - 1) + n \quad (n > 1) \quad (R)$$

(c) We can use the recurrence system to give the postcondition formally.

function $INTSUM(n \text{ in Int})$ **return in Int**

pre $n > 0$.

post $INTSUM(1) == 1$.

$(n > 1) \Rightarrow (INTSUM(n) == INTSUM(n - 1) + n)$.

Activity 1.3

- Give a recursive implementation of the function $INTSUM$.
- Determine the value returned for $INTSUM(4)$ by your implementation. Show the steps in the evaluation in the style shown in Example 1.2, indicating which statement in the implementation is being used at each step.

Discussion

(a) The postcondition leads naturally to recursive code, as below.

```
function INTSUM(n)
{ //recSum
  if (n == 1) then
  {
    return 1                //(S)
  }
  else
  {
    return INTSUM(n-1) + n    //(R)
  }
}
```

(b) The evaluation of $INTSUM(4)$ can be set out as follows.

$$\begin{aligned}
 INTSUM(4) &= INTSUM(3) + 4 && \text{(by R)} \\
 &= (INTSUM(2) + 3) + 4 && \text{(by R)} \\
 &= ((INTSUM(1) + 2) + 3) + 4 && \text{(by R)} \\
 &= ((1 + 2) + 3) + 4 && \text{(by S)} \\
 &= (3 + 3) + 4 \\
 &= 6 + 4 = 10.
 \end{aligned}$$

As before, repeated applications of (R) will lead to an eventual use of (S). The recursion then stops, and the result can be calculated.

An iterative implementation of the function $INTSUM$ is quite straightforward to code. However, in this case there is an even more straightforward approach. There is a very simple formula that gives the same values as the recurrence system for $INTSUM$, which is

$$INTSUM(n) = (n \times (n + 1)) / 2.$$

For example,

$$INTSUM(5) = (5 \times (5 + 1)) / 2 = 30 / 2 = 15.$$

Such a formula is known as a **solution** of the recurrence system. Not all recurrence systems have such a solution. In Units 14 and 15 we will discuss further how we can establish such solutions for recurrence systems.

Once we know there is such a formula, we can give a simple implementation.

```
function INTSUM(n)
{
  return DIV(n*(n+1),2)
}
```

There is no reason why this formula should be obvious!

In a recurrence system (such as that given for $INTSUM$ in the discussion of Activity 1.2) we have referred to (S) as the stopping condition. This reflects a recursive view. Equation (S) may also be referred to as the **starting condition** or **initial condition**. These terms reflect an iterative view — we have seen that a function specified through a recurrence system can be implemented either recursively or iteratively.

Aside

The values returned by *INTSUM* are known as **triangular numbers**. (To see why, count the dots in Figure 1.1.) Triangular numbers have many fascinating properties and appear in recreational books on numbers. (See for example <http://www.shyamsundergupta.com/triangle.htm>.) They also have a number of applications, for example in calculating the depreciation of an asset, packing problems, network connections and game theory.

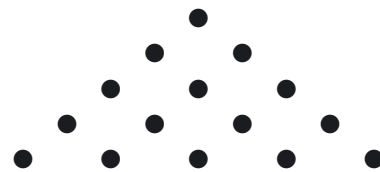


Figure 1.1 There are
 $1 + 2 + 3 + 4 + 5 = 15$ dots.

End of Aside

Our next example concerns the multiples of a given integer. This example would be of relevance in a situation where we are in the process of developing the data type **Int**. Suppose that so far only addition is available, and we want to give a specification and implementation of multiplication.

We can, for example, write the multiples of the integer 3 as follows.

3, 6, 9, 12, 15, 18, ...

The general term here is $3 \times n$. However, we cannot use this formula as a means of calculation in this case, because we are assuming here that the operation of integer multiplication is not yet available.

Another way to think about these multiples of 3 is in terms of addition: each multiple is formed by adding 3 to the previous one.

Activity 1.4

Let *MULTOFTHREE* be a function that has input n , where $n > 0$ is an integer, and returns $3 \times n$.

- Write down a recurrence system for *MULTOFTHREE*.
- Give a recursive implementation of *MULTOFTHREE* that uses only addition and subtraction of integers (not multiplication).

Discussion

- To find *MULTOFTHREE*(1), we have only a single 3 to add up:

$$\text{MULTOFTHREE}(1) = 3. \quad (\text{S})$$

To find *MULTOFTHREE*(n), we can add 3 to *MULTOFTHREE*($n - 1$):

$$\text{MULTOFTHREE}(n) = \text{MULTOFTHREE}(n - 1) + 3 \quad (n > 1). \quad (\text{R})$$

- Suitable code is given below.

```
function MULTOFTHREE(n)
{ //recMult
  if (n == 1) then
  {
    return 3                //(S)
  }
  else
  {
    return MULTOFTHREE(n-1) + 3  //(R)
  }
}
```

We could generalise this approach to multiplication by replacing 3 by a general integer. This shows that multiplication can be implemented by addition.

Aside

We could go back further. Suppose we have just two functions on integers: $NEXT(n) = n + 1$, and $PREV(n) = n - 1$. We can then implement addition recursively. To implement $ADD(n, m) = n + m$, we can use the following.

Here we consider functions whose domain is the integers n , where $n \geq 0$.

$$ADD(n, 0) = n$$

$$ADD(n, m) = NEXT(ADD(n, PREV(m))) \quad (m > 0)$$

Indeed, recursion can be used in defining what we mean by an integer. To do so is a subtle task, appropriate for an advanced course in mathematics. But it is worth noting here that recursion plays a crucial role in laying a formal foundation for the integers.

End of Aside

Our final example for this section concerns the **Fibonacci numbers**, which we can define as follows.

The first and second Fibonacci numbers are both 1. Each subsequent Fibonacci number is the sum of the previous two Fibonacci numbers.

So the third Fibonacci number is $1 + 1 = 2$, the fourth is $1 + 2 = 3$, the next is $2 + 3 = 5$, and so on.

Activity 1.5

Calculate the first ten Fibonacci numbers.

Discussion The first ten Fibonacci numbers are given below.

1, 1, 2, 3, 5, 8, 13, 21, 34, 55.

Activity 1.6

Give a specification of a function FIB that inputs an integer n , where $n > 0$, and returns the n th Fibonacci number. Give the postcondition formally.

Discussion The given description is expressed in a way that strongly suggest a recurrence system. This time, the recursive part refers back to two previous values of FIB and, as a consequence, we need two stopping conditions.

function $FIB(n \text{ in Int})$ **return in Int**

pre $n > 0$.

post $FIB(1) == 1$. (S1)

$FIB(2) == 1$. (S2)

$(n > 2) \Rightarrow (FIB(n) == FIB(n - 1) + FIB(n - 2))$. (R)

The postcondition again readily leads to a recursive implementation.

```

function FIB(n)
{ //recFib
if (n == 1) then
{ return 1 }           //(S1)
else
{
  if (n == 2) then
  { return 1 }         //(S2)
  else
  {
    return FIB(n-1) + FIB(n-2)  //(R)
  }
}
}

```

In this case, the two stopping conditions both return the same value, and so could be combined into a single condition.

Activity 1.7

Determine the value returned for $FIB(5)$ by *recFib*. Indicate which statement in the implementation is used at each step.

Discussion Following the self references is rather more complicated in this case.

$$\begin{aligned}
 FIB(5) &= FIB(4) + FIB(3) && \text{(by R)} \\
 &= (FIB(3) + FIB(2)) + (FIB(2) + FIB(1)) && \text{(using R twice)} \\
 &= ((FIB(2) + FIB(1)) + FIB(2)) + (FIB(2) + FIB(1)) && \text{(by R)} \\
 &= ((1 + FIB(1)) + 1) + (1 + FIB(1)) && \text{(by S2)} \\
 &= ((1 + 1) + 1) + (1 + 1) && \text{(by S1)} \\
 &= (2 + 1) + 2 = 5.
 \end{aligned}$$

There is a marked contrast between the amount of effort required in the calculations of the Fibonacci numbers performed in Activity 1.5 and Activity 1.7. Working by hand, it is much less effort to start at the front and go on calculating Fibonacci numbers until you reach the one you want, rather than working recursively. In general, computers perform calculations quickly and have plenty of storage capacity, but in this case the use of a recursive implementation stretches a machine's capability even for fairly small values of n . The algorithm *recFib* is extremely inefficient.

With FIB specified as in Activity 1.6, it is easy to see that the implementation *recFib* will be correct. But in such a case, practical considerations lead to a need to find an implementation that is iterative rather than recursive.

We will examine the idea of efficiency more carefully in Unit 15, but it is useful to make a couple of points here. In Unit 15, we shall associate a measure of the time taken to execute an implementation in terms of the size of the input. For FIB , the input is the integer n . It turns out that the time to calculate $FIB(n)$ using *recFib* grows exponentially (like the function 2^n , for example). We say that this implementation is “of exponential order”. In general, any implementation of exponential order is regarded as too inefficient to be of use. This is certainly the case for an input of any significant size. (We would not recommend trying to use the implementation *recFib* in the WorkPad for a value of n greater than about 28.)

Such an implementation is given in Exercise 1.3.

However, such implementations may be of use in certain circumstances, depending on the size of the input that has to be dealt with. For instance, the fractal implementations for *SPIKE* and *BUMP* are also of exponential order, but this is not a problem because we use only small values of n as input. (The curves that these functions produce soon reach a limit of screen resolution for n in the region of 5 to 10, after which further recursive calls no longer change the diagram as displayed on a screen.)

Aside

Leonardo Pisano is better known by his nickname Fibonacci. His famous book *Liber abaci* was published in 1202. A problem in the third section of this book led to the introduction of the Fibonacci numbers for which Fibonacci is best remembered today.

A certain man put a pair of rabbits in a place surrounded on all sides by a wall. How many pairs of rabbits can be produced from that pair in a year if it is supposed that every month each pair begets a new pair which from the second month on becomes productive?

Fibonacci numbers have applications in the field of computer graphics. In particular they appear in a technique called linear pixel shuffling to transform a greyscale image to a black-and-white image. They also have many applications in science.

See <http://www-gap.dcs.st-and.ac.uk/~history/Mathematicians/Fibonacci.html>

See Anderson, P. G. "Linear pixel shuffling for image processing: an introduction." *Journal of Electronic Imaging* April 1993, pp. 147–154 (also at <http://www.cs.rit.edu/~pga/darvas.pdf>).

For applications in science see, for example, Sutton, C. "Sunflower spirals obey laws of mathematics." *New Scientist*, 18 April 1992, p. 16 (or visit the web page http://www.cs.rit.edu/~pga/Fibo/fact_sheet.html).

End of Aside

Objectives for Section 1

After studying this section you should be able to do the following.

- ☐ Understand the terminology: stopping condition; recursive call.
- ☐ Understand what is meant by a recurrence system, and how such a recurrence system may be used to specify a suitable function with signature **Int** $\rightarrow$ **Int**.
- ☐ For a function F given by a recurrence system, calculate $F(n)$ for a given small value of n .
- ☐ For a suitable function described informally, determine a recurrence system to express the postcondition formally.
- ☐ For a function specified in terms of a recurrence system, give a recursive implementation.
- ☐ Given a recursive implementation, follow the execution for a given input, showing which statement is used at each step in the evaluation.
- ☐ Understand that a recursive implementation may be very inefficient.

Exercises on Section 1

Exercise 1.1

A function T is specified recursively below.

function $T(n \text{ in Int})$ **return in Int**

pre $n \geq 1$.

post $T(1) == 2$.

$(n \geq 2) \Rightarrow (T(n) == 2 \times T(n-1) + 3)$.

- (a) Give a recursive implementation of T . Show which statement gives the stopping condition.
- (b) Use your implementation to evaluate $T(4)$, showing which statement is used at each step.

Exercise 1.2

The function *POWEROFFIVE* has as input an integer n , where $n \geq 0$, and returns 1 if n is 0 and 5^n if $n > 0$.

- Give a recurrence system satisfied by *POWEROFFIVE*(n).
- Give a formal specification of *POWEROFFIVE*.
- Give a recursive implementation of *POWEROFFIVE*.

Recall that 5^n means $5 \times 5 \times 5 \times \dots \times 5$, where there are n 5s.

Exercise 1.3

A non-recursive implementation for the function *FIB* is given below. Trace the variables after each execution of the **for** loop when the input is 6.

```
function FIB(n)
{
    //itFib
    var k in Int
    var fTwoBack in Int
    var fOneBack in Int
    var f in Int
    fTwoBack <-- 1
    fOneBack <-- 1
    if (n == 1) then
    { f <-- 1 }
    if (n == 2) then
    { f <-- 1 }
    if (n >= 3) then
    {
        for (k <-- 2 to n-1)
        {
            f <-- fTwoBack + fOneBack
            fTwoBack <-- fOneBack
            fOneBack <-- f
        }
    }
    return f
}
```

2 Sequence-based types

In Section 1, we discussed how a recurrence system can be used to describe formally the postcondition of a suitable function on integers, and how such a description leads naturally to a recursive implementation of the function.

The implementation uses a recursive call: the function calling itself with a smaller input integer each time until, for a particular input, a stopping condition is reached and a specific output is returned for that particular call. It is only then that outstanding calls can be completed.

In this section we shall consider recursion on sequence-based types: **Stack**, **Vector** and **mString**. In general, a recursive implementation of a function will have the following features.

Requirements for valid recursion

1. *A stopping condition* For some specific input(s) the function will return a particular value.
2. *A recursive call* For inputs other than those of (1) the function calls itself with a different input.
3. *Termination* For all permitted inputs, the stopping condition must be reached after a finite number of recursive calls.

In the examples we looked at in Section 1, an integer input was decreased by 1 each time and the stopping condition occurred when a specific input such as 0, 1 or 2 was reached.

In using recursion on the types **Stack**, **Vector** and **mString**, we again need a stopping condition and a recursive call that eventually takes us to the stopping condition. Now, for an object of each of these types, we can associate an integer with that object — the integer which gives the number of items in the collection. For a stack this integer is given by the function *SIZE*, for a vector by the method *size*, and for a string by the method *length*. In each case, we need a recursive call that changes the associated integer so that it approaches a stopping value. For example, the stopping condition might relate to a stack containing 1 item (size 1), and each recursive call reduces the size of the input stack (until its size reaches 1).

One can have recursion where an integer *increases* at each step until a stopping value is reached, but such examples are less usual.

2.1 Recursion on stacks

To illustrate the way of thought that leads to a recursive description, we will consider the function below.

function *ADDONE*(*s* in Stack of Int) **return** in Stack of Int

pre true.

post The returned value is the stack formed by adding 1 to each number in the stack *s*. For example, if *s* is [4, 10, 7] then the returned value is the stack [5, 11, 8].

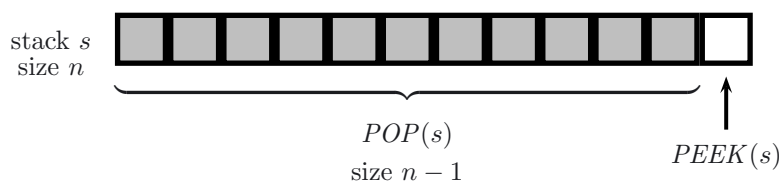


Figure 2.1 A stack, with *POP* and *PEEK* illustrated.

We can picture a stack as in Figure 2.1. In a stack, there is only one element that can be readily accessed — the top element, which is returned by the type function *PEEK*. The stack *POP(s)* has size one less than that of *s*. This suggests that we should:

- “process” the accessible item, *PEEK(s)*;
- use a recursive call to *POP(s)*.

For the function *ADDONE*, the “process” is to add 1 to each element. We can add 1 to the top element by forming *PEEK(s) + 1*. We could then return this “processed” element to the stack by using *PUSH* to put it back onto *POP(s)*. But just pushing *PEEK(s) + 1* onto *POP(s)* does not solve the problem of adding 1 to all the elements in the stack. We still have to add 1 to all the elements in the stack except the top one.

However, we *would* have achieved the required result if the stack $POP(s)$ had somehow already had 1 added to each of its elements. But this is precisely what $ADDONE$ is specified to do. In other words, we would like the stack $POP(s)$ to have first been processed by using the function $ADDONE$.

Now $ADDONE(POP(s))$ is a stack with 1 added to each of its elements. Suppose that we push $PEEK(s) + 1$ onto this. This will give $ADDONE(s)$.

$$ADDONE(s) == PUSH(PEEK(s) + 1, ADDONE(POP(s))) \quad (R)$$

This expression contains a recursive call of the function $ADDONE$ — and in it, $ADDONE$ is applied to the stack $POP(s)$.

Now $POP(s)$ has size 1 less than that of s , so each application of (R) will call $ADDONE$ on a smaller stack. The process of self-reference will continue until we reach a stopping condition — which we have not yet supplied. In this case the process cannot go on indefinitely, since we will eventually reach a stack of size 0 — the empty stack. This is the natural case to consider as a stopping condition for $ADDONE$. The empty stack has no elements in it, and the process of adding 1 to each of the non-existent elements will produce another empty stack.

We cannot have a stack whose size is a negative integer!

Expressed formally, the stopping condition can be written as follows.

$$\text{If } SIZE(s) == 0 \text{ then } ADDONE(s) = s \quad (S)$$

We can combine (R) and (S) to give a formal statement of the postcondition of $ADDONE$.

function $ADDONE(s \text{ in Stack of Int})$ **return in Stack of Int**

pre true.

post $(SIZE(s) == 0) \Rightarrow (ADDONE(s) == s)$.

$(NOT(SIZE(s) == 0)) \Rightarrow (ADDONE(s) == PUSH(PEEK(s) + 1, ADDONE(POP(s))))$.

This specification leads naturally to the recursive implementation below. (The implications in the formal postcondition above suggest code in the form “**if ... then ... else.**”)

```
function ADDONE(s)
{
    // recAddOne
    if (SIZE(s) == 0) then
    {
        return s // (S)
    }
    else
    {
        return PUSH(PEEK(s) + 1, ADDONE(POP(s))) // (R)
    }
}
```

The way we have designed this code provides strong reasons to be confident that it should be correct; but let us test it by following its execution for a particular input.

Activity 2.1

Use *recAddOne* to evaluate $ADDONE(s)$ when s is the stack $[4, 10, 7]$. Indicate which of the statements (S) or (R) is used at each step.

Discussion The stack $[4, 10, 7]$ has size 3, which is not 0, so the first call uses (R) to give:

$$ADDONE([4, 10, 7]) = PUSH(7 + 1, ADDONE([4, 10])). \quad (1)$$

(Note that $PEEK([4, 10, 7]) = 7$ and $POP([4, 10, 7]) = [4, 10]$.)

The stack $[4, 10]$ has size 2, which is not 0, so the next call again uses (R), to give:

$$ADDONE([4, 10]) = PUSH(10 + 1, ADDONE([4])). \quad (2)$$

The stack $[4]$ has size 1, which is not 0, so the next call again uses (R), to give:

$$ADDONE([4]) = PUSH(4 + 1, ADDONE([])). \quad (3)$$

The recursive call is now $ADDONE([])$. This has the empty stack (size 0) as input, and so uses (S), to give:

$$ADDONE([]) = [].$$

The recursion now stops, and the outstanding calls of $ADDONE$ in (1), (2) and (3) can then be evaluated.

We can lay out the full evaluation more briefly, as follows.

$$\begin{aligned} ADDONE([4, 10, 7]) &= PUSH(7 + 1, ADDONE([4, 10])) && \text{(by R)} \\ &= PUSH(7 + 1, PUSH(10 + 1, ADDONE([4]))) && \text{(by R)} \\ &= PUSH(7 + 1, PUSH(10 + 1, PUSH(4 + 1, ADDONE([])))) && \text{(by R)} \\ &= PUSH(7 + 1, PUSH(10 + 1, PUSH(4 + 1, []))) && \text{(by S)} \\ &= PUSH(7 + 1, PUSH(10 + 1, [4 + 1])) \\ &= PUSH(7 + 1, [4 + 1, 10 + 1]) \\ &= [4 + 1, 10 + 1, 7 + 1] \end{aligned}$$

We have left the output in this form so that it is clear that this stack has been formed by adding 1 to each element of the input stack. We can complete the arithmetic to get the output:

$$ADDONE([4, 10, 7]) = [5, 11, 8].$$

A possible point of confusion in such an evaluation is that each time the function is called the input is given the same identifier — s in *recAddOne* — even though each call refers to a different value for s . In the discussion above, the placeholder s refers to the stack $[4, 10, 7]$ during the evaluation of (1). Having got that far, $ADDONE([4, 10])$ must be found. During that process, which leads to (2), s refers to the stack $[4, 10]$. In the next step, leading to (3), s refers to the stack $[4]$. We will not discuss how a compiler may keep track of such references. Just note that each call of the code implementing $ADDONE$ works with its own stack, even if they all have the identifier s .

This discussion of Activity 2.1 shows that the implementation *recAddOne* gives the correct output for one particular input. It is possible to show that the recursive implementation does indeed give the correct output for all possible stack inputs (i.e. it is a correct implementation), but we shall not discuss this. The way such code is designed provides a strong indication that it is likely to be correct.

The recursive implementation in *recAddOne* is quite different to an iterative approach. In an iterative approach, we would add 1 to the each element of the input stack in turn. This is more awkward, given the limitations of the type functions available on stacks.

In such an example there is more of an issue in being certain that the formal and informal versions of the semantics are equivalent than in being sure that the recursive implementation correctly evaluates the function defined by the formal version of the specification.

Activity 2.2

Give an iterative implementation of *ADDONE*, using a **for** loop.

Concentrate on the general strategy that is required. Do not spend a long time on the details of the coding.

Discussion An implementation is given below.

```
function ADDONE(s)
{ //itAddOne
  var toDo in Stack of Int
  var soFar in Stack of Int
  toDo <-- s                                //(1)
  for (i <-- 1 to SIZE(s))
  {
    soFar <-- PUSH(PEEK(toDo) + 1, soFar)    //(2)
    toDo <-- POP(toDo)
  } // soFar now holds the output stack in reverse order
  for (i <-- 1 to SIZE(s))
  { // pass each item in turn from soFar to toDo
    toDo <-- PUSH(PEEK(soFar), toDo)
    soFar <-- POP(soFar)
  } // this reverses the order of the items again
  return toDo
}
```

In statement (1), the stack *toDo* is set to the input stack *s*. When the first **for** loop is entered, the stack *soFar* is in the default state, and so is empty.

In the first **for** loop, each item in the stack *s* has 1 added to it, in statement (2). The “processed” elements are stored in *soFar*. As the items are moved one at a time from *toDo* to *soFar*, the order of the items in the stack is reversed. The items are reversed again by the second **for** loop, so restoring their original order.

This iterative implementation is cumbersome in comparison to the recursive implementation. Because we can only insert and remove elements at one location in a stack (at the top), recursive implementations for functions on stacks are often easier to design and code than iterative implementations.

Activity 2.3

A function *SUMSTACK* is specified below.

function *SUMSTACK*(*s* in Stack of Int) **return** in Int

pre true.

post The returned value is the sum of the integers in the stack *s*.

For example, *SUMSTACK*([4, 2, 11]) = 17.

- Using recursion, express the postcondition formally.
- Give a recursive implementation.
- Use your recursive implementation in part (b) to evaluate *SUMSTACK*(*s*) when *s* is the stack [4, 2, 11], showing at each step whether the recursive call or the stopping condition is used.

Discussion We can develop a recursive description for $SUMSTACK(s)$ as follows.

1. Suppose that we know the result of applying $SUMSTACK$ to $POP(s)$.
2. Identify the single element that we can easily access.
3. Write down an expression adding this single item to the sum of the integers in $POP(s)$.
4. Identify the stopping condition required.

For a stack, we can only inspect the top element (shown on the right of the sequence), which we do with the type function $PEEK$. So $PEEK(s)$ will give us the first value in the required sum of all the integers in s .

If we apply the function $SUMSTACK$ to the stack $POP(s)$, then we get the sum of all the elements in the stack $POP(s)$. So this sum is given by $SUMSTACK(POP(s))$. To find the sum of all the integers in s , we want to add this to $PEEK(s)$. So we have

$$SUMSTACK(s) = PEEK(s) + SUMSTACK(POP(s)). \quad (R)$$

Applying POP repeatedly will eventually lead us to an empty stack. (Note that the precondition is **true**, so this case is in the domain of $SUMSTACK$.) The empty stack contains no integers, and the sum of no numbers is zero. So the stopping condition is

$$SUMSTACK([]) = 0. \quad (S)$$

(a) The statements (R) and (S) provide what is needed for a formal recursive postcondition.

post ($SIZE(s) == 0 \Rightarrow (SUMSTACK(s) == 0)$.
 $(NOT(SIZE(s) == 0)) \Rightarrow (SUMSTACK(s) == PEEK(s) + SUMSTACK(POP(s)))$).

(b) This leads to the following recursive implementation.

```
function SUMSTACK(s)
{
  if (SIZE(s) == 0) then
  {
    return 0
  }
  else
  {
    return PEEK(s) + SUMSTACK(POP(s))
  }
}
```

(c) With the input stack $[4, 2, 11]$, we can set out the evaluation as below.

$$\begin{aligned}
SUMSTACK([4, 2, 11]) &= 11 + SUMSTACK([4, 2]) && \text{(by R)} \\
&= 11 + (2 + SUMSTACK([4])) && \text{(by R)} \\
&= 11 + (2 + (4 + SUMSTACK([]))) && \text{(by R)} \\
&= 11 + (2 + (4 + 0)) && \text{(by S)} \\
&= 11 + (2 + 4) = 11 + 6 = 17.
\end{aligned}$$

As an aside, note that although postconditions given in recursive form lead quite naturally to a recursive implementation, it is not necessary for a recursive postcondition to be implemented recursively. We have seen non-recursive implementations of various functions with recursive specifications, such as *ADDONE* and *FACTORIAL*.

2.2 Recursion on vectors

A vector is similar to a stack. Both have the same form of data — a sequence of items — and the methods of **Vector** enable us to: access the last element in a vector, to remove this last element, and to replace it with another. So for similar tasks, we would expect to be able to translate a recursive implementation for stacks into a corresponding implementation for vectors.

Of course, we also have other methods available to a vector.

For example, we should be able to draw on the design of the implementation of *SUMSTACK* given in the discussion of Activity 2.3 to implement a function (say *ADDALL*) to sum a vector of integers.

function *ADDALL*(*v* in **Vector** of **Int**) **return** in **Int**

pre true.

post The returned value is the sum of all the integers in *v*.

Nevertheless, there is one major difference between the type **Stack** and the class **Vector**. This key difference is exemplified in the difference between the type function *POP* and the method *removeLast*. The effect of each might be summarised by saying “the last item is removed from the sequence”. However, *POP* returns a value, while *removeLast* changes the state of the receiver and does not return a value. This difference needs to be reflected in code.

For a stack *s*, the expression *SUMSTACK*(*POP*(*s*)) is valid.

For a vector *v*, the expression *ADDALL*(*v.removeLast*()) is *not* valid.

Remember, *v.removeLast*() is not a vector. It is a statement that changes the state of *v*. To apply the function *ADDALL* to the changed vector we need two separate code statements: *v.removeLast* then *ADDALL*(*v*). In the second statement, the function *ADDALL* is applied to the vector that is obtained from the original state of *v* by removing the last element. The value returned by this function call will need to be assigned to a variable if it is not to be lost, as exemplified below (*n* is a variable of type **Int**).

```
v.removeLast() //remove the last element from v
n <-- ADDALL(v) //apply function to the changed vector v
```

EXAMPLE 2.1

We give below a recursive implementation of *ADDALL*. The recursion is based on similar reasoning to that in Activity 2.3. Since we are now dealing with a vector, we must of course use the methods of **Vector**. But note the less obvious differences that follow from the issues discussed in the text above.

The local variable *n* is used to hold the value to be returned.

In (1), *clone* is used to make a copy of the input object *v*. (We need to use this to avoid changing the state of the input vector when *ADDALL* is called in user code.)

In (2), *n* is given the value 0 — the stopping value — if *v* is empty.

In (3), we save a copy of the last integer in *v* before we remove it from the vector *temp*.

In (4), we change the state of *temp* by removing the last element.

In (5), we apply the recursive call on the changed (shortened) vector *temp*, and add this to the last element of *v* that was stored in *n*.

In (6), we return the value of *n*, which will be determined by (2) if *v* is empty and by (5) if *v* is not empty.

```

function ADDALL(v)
{
  var temp in Vector of Int
  var n in Int
  temp <-- v.clone()           //(1)
  if (temp.size() == 0) then
  {
    n <-- 0                     //(2) stopping value
  }
  else
  {
    n <-- temp.last()           //(3)
    temp.removeLast()           //(4)
    n <-- n + ADDALL(temp)      //(5) recursive call
  }
  return n                     //(6)
}

```

In the example above, the **else** clause contains three statements (one being the recursive call). In the case of a stack, only one statement was needed. The use of state-changing methods makes it impracticable to combine these into a formal recursive postcondition for this function. While it is possible to implement functions with an input of type **Vector** recursively, to do so is complicated by the state-changing methods of that class. Since we have access to all elements in a vector, it is often easier to implement a function on vectors iteratively.

Activity 2.4

Use a **for** loop to implement the function *ADDALL*.

Discussion We can use the method *at* to extract each element in turn, and keep a running total of the sum of these integers.

```

function ADDALL(v)
{
  //itAddAll
  var i in Int
  var sum in Int
  sum <-- 0
  for (i <-- 1 to v.size())
  {
    sum <-- sum + v.at(i)
  }
  return sum
}

```


2.3 Recursion on strings

Because of the nature of the methods of **Vector**, recursion is less often useful for vectors than for stacks. We now consider another form of data based on a sequence. Recall that a string is a sequence of characters. Now the methods of **mString** return a value rather than changing the state of the receiving object. This avoids the technical problems with coding recursion that are encountered with **Vector**, and so we can hope to construct recursive specifications for functions involving this class. First, we summarise what we have seen so far about recursion for collection types.

See Unit 6.

Recursion for sequence-based collection types

1. We look for easy access, deletion and insertion at one end of the collection.
2. We then process the element at this one end in some way and use a recursive call to process the rest of the collection (i.e. everything but the end element) in a similar manner.
3. We look for a stopping condition (a return value that can be determined immediately for certain special cases — usually when the collection is small, often when empty).

Together, points 2 and 3 must ensure that an output is determined for all permitted inputs.

For the class **mString**, we can readily access the character at any position in a string (using *charAt*), but we can only easily add a character at the end of the string (using the method *charAppend*). To find the string resulting from deletion of the last character, we can use either of the methods that return a substring (*substring* or *substr*).

Activity 2.5

Give a recursive implementation of the function *TOUPPER*, specified below.

function *TOUPPER*(*s* in **mString**) **return** in **mString**

pre true.

post The returned value is the same as the string *s*, except that every lower-case letter character in *s* has been changed to the corresponding upper-case letter character. (If *s* is empty, the returned value is an empty string.)

An iterative implementation of *TOUPPER* was given in Unit 6, p. 39.

Discussion The idea behind the implementation is as follows.

If the string *s* is empty, return an empty string.

If the string *s* is not empty, first apply *TOUPPER* to the string obtained by deleting the last character in *s*, and store the result in *u*. If the last character in *s* is not a lower-case letter, then add it unchanged to the end of *u*. If it is a lower-case letter, first change it to the corresponding upper-case letter, and then add this to the end of *u*. Then return *u*.

An implementation is given below.

Note that the default value for **mString** is the empty string, so *u* is given the required stopping value by default in statement (1).

In (2), *u* is set to the substring of *s* starting at index position 1 (the first character) and whose length is one less than the length of *s*.

In (3), we apply *TOUPPER* to this shortened string.

In (5), we add the last character of s to the end of u , changed to upper-case by (4) if necessary.

```
function TOUPPER(s)
{
  var u in mString           //(1)
  var c in Char
  if (NOT(s.length() == 0)) then
  {
    u <-- s.substr(1,s.length() - 1)   //(2)
    u <-- TOUPPER(u)                   //(3)
    c <-- s.charAt(s.length())
    if (('a' <= c) /\ (c <= 'z')) then
    {
      c <-- CHR(ASC(c) - 32)           //(4)
    }
    u <-- u.charAppend(c)              //(5)
  }
  return u
}
```

Since the stopping condition is for an empty string, it is essential that we use *substr* (rather than *substring*) in (2). The method *substr* will return an empty string when the second parameter is 0 (see Unit 6, p. 33).

By using *substr*, we can see immediately that the recursive call of *TOUPPER* is being applied to a string whose length has been reduced.

We now look at a more complicated task: to reverse a string. A corresponding function, *STRREV*, is specified below. For example:

STRREV("test string") = "gnirts tset".

function *STRREV*(s in mString) **return** in mString

pre s contains at least one character.

post The returned value is the string obtained by reversing the order of the characters in the string s .

Figure 2.2 illustrates a way of viewing this task that leads towards a recursive implementation. In the reversed string, the first character of s will have been moved to the end. We also need to reverse the remainder of s . This can be achieved by a recursive call of *STRREV* on the remainder of s — the string *substr*(2, $s.length() - 1$). After this recursive call, we can use *charAppend* to add the first character of s to the end of the reversed string.

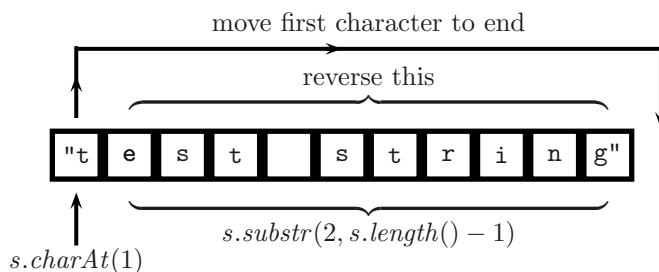


Figure 2.2 Reversing a string.

Activity 2.6

Give a recursive implementation for the function *STRREV*.

Discussion Here, the stopping condition will relate to a string of length 1. In that case, we can return a copy of the input string, since reversing a string containing exactly one character will not change it.

A recursive implementation is given below.

At (2), u is set to the string formed by removing the first character from s .

At (3), this reduced string is reversed by the recursive call.

At (4), the first character of s is placed in c , and this is added to the end of the reversed string in (5).

At each recursive call the length of the string is decreased by 1. Eventually, this will lead to a call of *STRREV* for a string of length 1. Then the recursion will terminate, using the stopping condition in (1).

```
function STRREV(s)
{
  // recStrRev
  var u in mString
  var c in Char
  if (s.length() == 1) then
  {
    u <-- s.clone()           //(1)
  }
  else
  {
    u <-- s.substr(2,s.length() - 1)   //(2)
    u <-- STRREV(u)                 //(3)
    c <-- s.charAt(1)                //(4)
    u <-- u.charAppend(c)            //(5)
  }
  return u
}
```

In the code in the preceding discussion, we used a series of statements reflecting the various steps in the design of the implementation. These could be amalgamated into fewer statements, but that would lead to less clear code. However, if we want to use recursion to provide a formal statement of the postcondition of *STRREV*, then we need to combine the steps into a single expression. This is done below.

function *STRREV*(s in mString) **return** in mString

pre $s.length() > 0$.

post $(s.length() == 1) \Rightarrow (STRREV(s) == s)$.

$(s.length() > 1) \Rightarrow (STRREV(s) ==$

$(STRREV(s.substr(2, s.length() - 1))).charAppend(s.charAt(1)))$.

In an example such as this, we end up with a complicated formal expression.

Suppose that the implementation *recStrRev* given in the discussion of Activity 2.6 is used to evaluate *STRREV*(s) when s is the string “xyz”. We can follow this evaluation as below.

When s is “xyz”:

$s.length() == 1$ is **false**;
 then u is set to “yz”;
 then u is set to $STRREV(\text{“yz”})$ (this recursive call must remain pending);
 then c is set to ‘x’;
 then u is set to $STRREV(\text{“yz”}).charAppend(\text{‘x’})$;
 this value will be returned as $STRREV(\text{“xyz”})$.

When s is “yz”:

$s.length() == 1$ is **false**;
 then u is set to “z”;
 then u is set to $STRREV(\text{“z”})$ (this recursive call must remain pending);
 then c is set to ‘y’;
 then u is set to $STRREV(\text{“z”}).charAppend(\text{‘y’})$;
 this value will be returned as $STRREV(\text{“yz”})$.

When s is “z”:

$s.length() == 1$ is **true**;
 then u is set to “z” by (1),
 and this value is returned as $STRREV(\text{“z”})$.

Then

$$\begin{aligned} STRREV(\text{“yz”}) &= STRREV(\text{“z”}).charAppend(\text{‘y’}) \\ &= \text{“z”}.charAppend(\text{‘y’}) = \text{“zy”}. \end{aligned}$$

Then

$$\begin{aligned} STRREV(\text{“xyz”}) &= STRREV(\text{“yz”}).charAppend(\text{‘x’}) \\ &= \text{“zy”}.charAppend(\text{‘x’}) = \text{“zyx”}. \end{aligned}$$

EXAMPLE 2.2

The evaluation of $STRREV(\text{“xyz”})$ using *recStrRev* can be summarised as follows.

$$\begin{aligned} STRREV(\text{“xyz”}) &= STRREV(\text{“yz”}).charAppend(\text{‘x’}) && \text{(by 5)} \\ &= (STRREV(\text{“z”}).charAppend(\text{‘y’})).charAppend(\text{‘x’}) && \text{(by 5)} \\ &= (\text{“z”}.charAppend(\text{‘y’})).charAppend(\text{‘x’}) && \text{(by 1)} \\ &= \text{“zy”}.charAppend(\text{‘x’}) = \text{“zyx”}. \end{aligned}$$

Objectives for Section 2

After studying this section you should be able to do the following.

- ☐ For a suitable function on **Stack** or **mString**, give a recursive implementation.
- ☐ For a suitable function on **Stack** or **mString**, use recursion to express the postcondition formally.
- ☐ For a suitable function on **Vector**, give a recursive implementation. Be aware here of the difference needed in coding because key methods of this class change the state of a vector rather than returning a value.
- ☐ For a particular input, determine the returned value obtained by using a given recursive implementation, documenting which statements are called and how these produce the returned value.

Exercises on Section 2

Exercise 2.1

Explain why the following recursive code is not a valid implementation of a function F that has an integer n as input, where $n > 0$.

```
function F(n)
{
  if (n == 1) then
  {
    return 1
  }
  else
  {
    return 1 + F(n-2)
  }
}
```

Exercise 2.2

Give a recursive implementation for the function *CLEARTO* specified below.

function *CLEARTO*(v in **Vector of Int**) **return** in **Vector of Int**

pre true.

post The returned value is a vector consisting of the elements in v from the first 0 in v to the end. If there is no 0 in v , then the empty vector is returned.

For example, *CLEARTO*([1, 2, 3, 0, 4, 5, 6]) is the vector [0, 4, 5, 6].

Exercise 2.3

A function F has two inputs s and t of type **Stack of Int**, and returns a value in **Stack of Int**. A recursive implementation of F , *recF*, is given below. Use *recF* to evaluate $F(s, t)$ when s is the stack [5, 3, 1] and t is the stack [4, 2], showing which of the lines (S1), (S2) or (R) is used at each step.

```
function F(s,t)
{
  //recF
  if (SIZE(s) == 0) then
  {
    return t                                //(S1)
  }
  else
  {
    if (SIZE(t) == 0) then
    {
      return s                                //(S2)
    }
    else
    {
      return PUSH(PEEK(s), PUSH(PEEK(t), F(POP(s), POP(t)))) // (R)
    }
  }
}
```

Exercise 2.4

A function *CODE* has an input of type **mString** and returns a value of type **mString**. A recursive implementation of *CODE*, *recCode*, is given below.

```
function CODE(s)
{
    // recCode
    var c in Char
    var u in mString
    if (s.length() == 0) then
    {
        u <-- s                                //(S)
    }
    else
    {
        c <-- s.charAt(s.length())
        c <-- CHR(ASC(c)+1)
        u <-- s.substr(1,s.length()-1)
        u <-- CODE(u).charAppend(c)           //(R)
    }
    return u
}
```

(a) Use *recCode* to evaluate *CODE*(*s*) when *s* is the string “abC”. Try to express your evaluation in a summary form like that in Example 2.2. Show which of statements (S) or (R) is used in each recursive call of *CODE*.

(b) Suggest a specification for the function *CODE* implemented by *recCode*, with the semantics given in English.

3 Binary trees

As we noted in Unit 8, a binary tree is an inherently recursive form of data. So it should be no surprise to you that these structures are often conveniently processed using recursive implementations. Now a binary tree does not have random access — the only element that is readily accessible is that at the root of the tree. The rest of the collection splits neatly into two parts: the left and the right subtrees, both of which are binary trees themselves. See Unit 8, p. 10.

3.1 Recursion on binary trees

For functions with binary tree inputs we can process the root element and then apply recursive calls to the left and right subtrees. For example, we mentioned the following example in the Introduction to this unit. Think about how you might implement a function whose input is a binary tree of integers, and which returns the sum of all the integers in the tree. To do it recursively, we can draw on the following insight.

If the tree is empty, the sum is 0.

If the tree is not empty, the required sum is: the root + the sum of the numbers in the left subtree + the sum of the numbers in the right subtree.

Activity 3.1

Give a specification of a function *SUMT* that inputs a binary tree of integers and returns the sum of all the integers in the tree. (Give the postcondition informally.) Then use recursion to give an implementation of *SUMT*.

Discussion A specification of *SUMT* is given below.

function *SUMT*(*t* in BinTree of Int) **return** in Int

pre true.

post The returned value is the sum of all the integers appearing at the nodes of *t*. (If *t* is empty then 0 is returned.)

The left subtree of a tree *t* is returned by *t.leftTree()* — recall that the method *leftTree* returns a value which is a binary tree. So the sum of the integers in the left subtree is given by the recursive call *SUMT*(*t.leftTree()*). Similarly, the sum of the integers in the right subtree will be given by the recursive call *SUMT*(*t.rightTree()*). So, if *t* is not empty, the sum of the integers at all the nodes is the sum of these two expressions plus the root node, i.e.

t.getRoot() + *SUMT*(*t.leftTree()*) + *SUMT*(*t.rightTree()*).

The stopping case is when *t* is empty, when we want to return 0. These ideas are incorporated into a recursive implementation of *SUMT* below.

```
function SUMT(t)
{  //recSumTree
  if (t.isEmpty()) then
  {
    return 0                                //(S)
  }
  else
  {
    return t.getRoot()+SUMT(t.leftTree())+SUMT(t.rightTree())  //(R)
  }
}
```

We can use the same approach to express the postcondition of *SUMT* formally, as follows.

function *SUMT*(*t* in BinTree of Int) **return** in Int

pre true.

post (*t.isEmpty()*) $\Rightarrow$ (*SUMT*(*t*) == 0)

($\neg$ (*t.isEmpty()*)) $\Rightarrow$

(*SUMT*(*t*) == *t.getRoot()* + *SUMT*(*t.leftTree()*) + *SUMT*(*t.rightTree()*))

It is usually not straightforward to replace a recursive implementation on binary trees by an iterative alternative.

Activity 3.2

By following the execution of the implementation *recSumTree*, evaluate *SUMT*(*t*) when *t* is the tree shown in Figure 3.1.

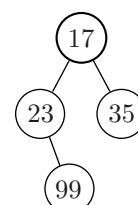


Figure 3.1

Discussion The input tree is shown in Figure 3.1, its left subtree in Figure 3.2(a) and its right subtree in Figure 3.2(b). The tree shown in Figure 3.2(b) consists of a single root node. Recall from Unit 8 (page 10) that we may write this tree as $(35 \ () \ ())$. This linear notation is convenient when writing out an evaluation such as this. For the input tree in Figure 3.1 we have, from (R):

$$SUMT(t) = 17 + SUMT(t1) + SUMT(t2), \quad (1)$$

where $t1$ is the tree in Figure 3.2(a), and $t2$ is the tree $(35 \ () \ ())$ shown in Figure 3.2(b). Now $t2$ is not empty, so we use (R) to get

$$\begin{aligned} SUMT(t2) &= 35 + SUMT(ET()) + SUMT(ET()) \\ &= 35 + 0 + 0 \quad (\text{using S twice}) \end{aligned}$$

$$\text{So } SUMT(t2) = 35. \quad (2)$$

Now consider the recursive call $SUMT(t1)$.

$$\begin{aligned} SUMT(t1) &= 23 + SUMT(ET()) + SUMT((99 \ () \ ())) && (\text{from R}) \\ &= 23 + 0 + SUMT((99 \ () \ ())) && (\text{using S}) \\ &= 23 + 0 + (99 + SUMT(ET()) + SUMT(ET())) && (\text{from R}) \\ &= 23 + 0 + (99 + 0 + 0) && (\text{using S twice}) \end{aligned}$$

$$\text{So } SUMT(t1) = 122. \quad (3)$$

We can now go back to Equation (1) and evaluate $SUMT(t)$ as:

$$\begin{aligned} SUMT(t) &= 17 + SUMT(t1) + SUMT(t2) && (\text{from (1)}) \\ &= 17 + 122 + 35 && (\text{using (2) and (3)}) \\ &= 174. \end{aligned}$$

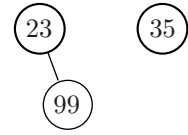


Figure 3.2

Recall that $ET()$ is an empty tree.

This example illustrates that, in a recursive implementation of a function whose input is a binary tree, we can expect there to be two smaller trees to process further: the left and right subtrees. Compare this to the situation with a stack input, where we usually expect to have a single smaller stack (returned by *POP*) to process further. A feature of recursion on binary tree is that we typically have to consider recursive calls on two subtrees.

Activity 3.3

A specification is given below of a function *NODECT*.

function *NODECT*(t in BinTree of X) return in Int

pre true.

post The returned value is the number of nodes in t .

For example, for the tree in Figure 3.1, $NODECT(t) = 4$.

- Give a recursive implementation of *NODECT*. You will need to consider the following steps.
 - For a non-empty tree t , describe how to find the value $NODECT(t)$ by considering t as consisting of: a root element, a left subtree and a right subtree.
 - Express formally the recursive call that processes each subtree of t .
 - Give a stopping condition that will terminate the recursion.
 - Finally, give the code for this entire implementation.
- Use recursion to express the postcondition of *NODECT* formally.

Discussion

(a) (i) If the tree is not empty, then 1 should be added to the returned value, to count the root node. The returned value will be:

$1 + (\text{the number of nodes in the left subtree}) + (\text{the number of nodes in the right subtree})$.

The second and third terms will be obtained using recursive calls of *NODECT*, applied to the each subtree.

(ii) The recursive calls can be expressed formally as:

$NODECT(t.\text{leftTree}())$ and $NODECT(t.\text{rightTree}())$.

(iii) The stopping condition will occur when the tree t is empty. The number of nodes in an empty tree is 0.

(iv) A recursive implementation of *NODECT* is given below.

```
function NODECT(t)
{
  if (t.isEmpty()) then
  {
    return 0
  }
  else
  {
    return 1 + NODECT(t.leftTree()) + NODECT(t.rightTree())
  }
}
```

(b) Below, the postcondition in the specification is expressed formally using recursion.

function *NODECT*(t in **BinTree of X**) **return in** **Int**

pre true.

post $(t.\text{isEmpty}()) \Rightarrow (NODECT(t) == 0)$.

$(\neg(t.\text{isEmpty}())) \Rightarrow$
 $(NODECT(t) == 1 + NODECT(t.\text{leftTree}()) + NODECT(t.\text{rightTree}()))$.

In the next example, we consider a function that returns a binary tree. This function, *ADDONET*, adds 1 to each number in a tree of integers

function *ADDONET*(t in **BinTree of Int**) **return in** **BinTree of Int**

pre true.

post The returned tree is formed by adding 1 to the integer at each node in t . If t is empty, then an empty tree is returned.

EXAMPLE 3.1

Below, we give a recursive implementation of *ADDONET*.

The three statements (R1)–(R3) in the recursive part build the output tree by: first setting the required root, then the required left subtree, then the required right subtree.

The stopping condition is for the empty tree. If t is empty, the **if** condition is **false**, and *temp* remains a copy of the empty tree — the required returned value in that case. So the stopping value is provided by default in statement (SV), and the stopping condition is the negation of the **if** condition in (1).

```
function ADDONET(t)
{
    // recAddOneT
    var temp in BinTree           //(SV)
    temp <-- t.clone()
    if (NOT(t.isEmpty())) then    //(1)
    {
        temp.setRoot(t.getRoot()+1)    //(R1)
        temp.setLeftTree(ADDONET(t.leftTree())) //(R2)
        temp.setRightTree(ADDONET(t.rightTree())) //(R3)
    }
    return temp
}
```

3.2 Traversing binary trees

In Unit 8 we discussed the in-order traversal of a binary tree. We observed that this is awkward to formalise without recursion, either as a strategy or as code. The essence of the idea of the in-order traversal is the following.

See Subsection 2.2 of Unit 8.

Form a sequence as follows.

1. Write down the in-order traversal of the left subtree, then write down the root, then write down the in-order traversal of the right subtree.
2. The in-order traversal of an empty tree is the empty sequence.

See Unit 8, p. 27.

This description is essentially recursive, as was noted in Unit 8. To express this approach as a recursive algorithm we will give an implementation of the function *IN_ORDER*, specified below. (We choose to give a returned value of type **VectorPlus**, because that class has a method for concatenating sequences, which we will use in the implementation.)

function *IN_ORDER*(t in **BinTree** of **X**) **return** in **VectorPlus** of **X**
pre **true**.

post The returned value is a vector consisting of the elements stored at the nodes of t , given in the order determined by an in-order traversal.

There is a small discrepancy with the way we specified the function *IN_ORDER* in Unit 8, where the returned value was of type **Vector**. In Unit 8 we were only concerned with the form of data being returned, not with how to implement the function.

We can make the following observations.

- The stopping case is for the empty tree, and in that case the returned value is the empty sequence.
- We can formalise recursive calls on each subtree as *IN_ORDER*($t.leftTree()$) and *IN_ORDER*($t.rightTree()$).
- The returned sequence is formed as: *IN_ORDER*($t.leftTree()$), then the root of t , then *IN_ORDER*($t.rightTree()$).

An implementation is given below. To combine the three parts of the returned sequence, we can use the methods *join* and *addLast* of the class **VectorPlus**. We need to recall that these methods change the state of the receiving vector.

See Unit 6, p. 27, for the specification of *join*.

```
function IN_ORDER(t)
{
    //recInOrder
    var v in VectorPlus          // (SV)
    if (NOT(t.isEmpty())) then
    {
        v.join(IN_ORDER(t.leftTree())) // process left subtree
        v.addLast(t.getRoot())         // add the root value
        v.join(IN_ORDER(t.rightTree())) // process right subtree
    }
    return v
}
```

Recall that the default state of a vector is empty, so v will be empty after (SV). If t is empty, the **if** condition will be false, and v will remain empty, and this value will be returned. This provides a stopping condition. If t is not empty, the code first sets v to the in-order traversal of the left subtree, then adds the root of the tree at the right-hand end of this, then concatenates the in-order traversal of the right subtree to this (again, adding it at the right-hand end of v).

Note the placement of the statement adding the root, which is between the two recursive calls. This means that the root is placed *between* the two sequences formed from the left subtree and the right subtree.

Aside

Within the execution of *recInOrder*, a variable vector v is defined in each call. Although each of these has the same identifier v , each one is local to its own call and so is a different variable (and is not visible within the other calls). This can lead to the generation of a large number of “pending” variables during execution of such code, which can be a problem when it is implemented for a large input.

End of Aside

Activity 3.4

Determine the value returned for $IN_ORDER(t)$ by the implementation *recInOrder* above, when t is the tree shown in Figure 3.1. For each recursive call, indicate which part of the code is used. Use (R) to refer to the recursive part of the implementation *recInOrder*, consisting of the three statements in the **if** clause, and use (S) to refer to the stopping case, which occurs when t is empty, and when the default value of **VectorPlus** — the empty vector — is returned.

In setting out the evaluation, we will use the infix notation $\oplus$ to show concatenation of sequences, and linear notation (see Unit 8, page 10) to show trees of depth 1.

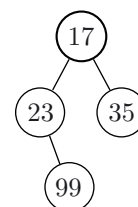


Figure 3.1

Discussion Let t_1 and t_2 denote the left and right subtrees of t , as shown in Figure 3.2.

From (R), we have

$$IN_ORDER(t) = IN_ORDER(t_1) \oplus [17] \oplus IN_ORDER(t_2). \quad (1)$$

Then, again using (R), we have

$$\begin{aligned} IN_ORDER(t_1) &= IN_ORDER(ET()) \oplus [23] \oplus IN_ORDER((99 () ())) \\ &= [] \oplus [23] \oplus IN_ORDER((99 () ())) && \text{(using S)} \\ &= [] \oplus [23] \oplus (IN_ORDER(ET()) \oplus [99] \oplus IN_ORDER(ET())) && \text{(by R)} \\ &= [] \oplus [23] \oplus ([] \oplus [99] \oplus []) && \text{(using S twice)} \\ &= [23, 99]. \end{aligned}$$

Now t_2 is the tree $(35 () ())$, so (1) becomes

$$\begin{aligned} IN_ORDER(t) &= [23, 99] \oplus [17] \oplus IN_ORDER((35 () ())) \\ &= [23, 99, 17] \oplus (IN_ORDER(ET()) \oplus [35] \oplus IN_ORDER(ET())) && \text{(by R)} \\ &= [23, 99, 17] \oplus ([] \oplus [35] \oplus []) && \text{(using S twice)} \\ &= [23, 99, 17, 35]. \end{aligned}$$

This is the in-order traversal of the tree t .

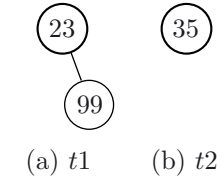


Figure 3.2

See Unit 8, p. 26, Strategy 2.3.

In Unit 8, we also discussed another form of tree traversal, the breadth first tree traversal (returned by the function *LEVEL_ORDER*). This is not naturally formulated recursively. There are two other forms of tree traversal that are often encountered. These can be implemented by small variations on the code in *recInOrder*. The feature that varies is where the root is added in relation to the recursive calls to the two subtrees. The in-order traversal has this call *between* these two recursive calls. In the **pre-order traversal** the root is added *before* both the recursive calls. In the **post-order traversal** the root is added *after* both the recursive calls.

A function *PRE_ORDER*, corresponding to the pre-order traversal, is specified below, and is followed by a recursive implementation.

function *PRE_ORDER*(t in **BinTree of X**) **return in VectorPlus of X**
pre true.

post The returned value is a sequence giving the values at the nodes of t in the order given by the pre-order traversal (discussed below).

```

function PRE_ORDER(t)
{
    //recPreOrder
    var v in VectorPlus                //(SV)
    if (NOT(t.isEmpty())) then
    {
        v.addLast(t.getRoot())          //add the root value
        v.join(PRE_ORDER(t.leftTree())) //process left subtree
        v.join(PRE_ORDER(t.rightTree())) //process right subtree
    }
    return v
}
  
```

We now examine an example of the effect of this implementation. In the evaluation we use (R) to refer to the recursive part of the implementation *recPreOrder*, consisting of the three statements in the **if** clause, and use (S) to refer to the stopping value, which occurs when t is empty, and is the default value of **VectorPlus** — the empty vector.

EXAMPLE 3.2

We will evaluate the value returned for $PRE_ORDER(t)$ by the implementation $recPreOrder$ above, when t is the tree shown in Figure 3.1. Again, let t_1 and t_2 to denote the left and right subtrees of t , as shown in Figure 3.2. From (R), we have

$$PRE_ORDER(t) = [17] \oplus PRE_ORDER(t_1) \oplus PRE_ORDER(t_2). \quad (1)$$

Then, again using (R), we have

$$\begin{aligned} PRE_ORDER(t_1) &= [23] \oplus PRE_ORDER(ET()) \oplus PRE_ORDER((99 \ (\) \ (\))) \\ &= [23] \oplus [] \oplus PRE_ORDER((99 \ (\) \ (\))) && \text{(using S)} \\ &= [23] \oplus [] \oplus ([99] \oplus PRE_ORDER(ET()) \oplus PRE_ORDER(ET())) && \text{(by R)} \\ &= [23] \oplus [] \oplus ([99] \oplus [] \oplus []) && \text{(using S twice)} \\ &= [23, 99]. \end{aligned}$$

Now t_2 is the tree $(35 \ (\) \ (\))$, so (1) becomes

$$\begin{aligned} PRE_ORDER(t) &= [17] \oplus [23, 99] \oplus PRE_ORDER((35 \ (\) \ (\))) \\ &= [17, 23, 99] \oplus ([35] \oplus PRE_ORDER(ET()) \oplus PRE_ORDER(ET())) && \text{(by R)} \\ &= [17, 23, 99] \oplus ([35] \oplus [] \oplus []) && \text{(using S twice)} \\ &= [17, 23, 99, 35]. \end{aligned}$$

This is the pre-order traversal of the tree t .

We can illustrate the pre-order tree traversal using a tree walk diagram, as in Figure 3.3. In the pre-order tree walk, we write down the item at each node when we pass the node *on the left*. For the tree t in Figure 3.3, we have

$$PRE_ORDER(t) = [11, 22, 44, 55, 33, 66, 77].$$

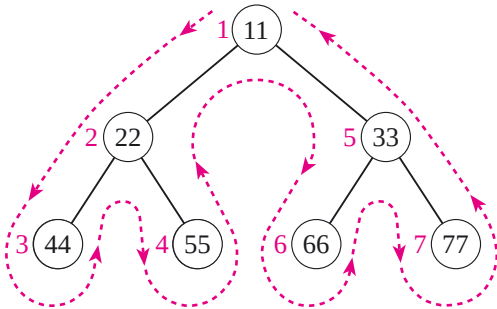


Figure 3.3 A pre-order tree traversal.

Earlier, we also mentioned the post-order tree traversal. The corresponding function is $POST_ORDER$. To obtain $POST_ORDER$, we write down: the sequence corresponding to the left subtree, then the sequence corresponding to the right subtree, then the root item. Thinking in terms of a tree walk diagram like that in Figure 3.3, we write down the item at each node when we pass the node *on the right*.

For the tree t shown in Figure 3.3:

$$POST_ORDER(t) = [44, 55, 22, 66, 77, 33, 11].$$

A tree walk diagram for an in-order tree walk is shown in Figure 2.10(a) of Unit 8.

For the in-order tree walk, we write down each node as we pass *beneath* the node.

3.3 Searching in a binary tree

Section 3 of Unit 8 discussed sorted trees. In particular, the binary search tree, or BST, was introduced there. We noted that a BST is a form of data well designed for searches, i.e. for determining whether a particular item appears in the collection. We now consider the process of searching for an item in a tree. We first consider searching in a general tree, i.e. an arbitrary binary tree (which need not be a BST).

See Unit 8, p. 34.

function *INTREE*(*t* in **BinTree** of **X**, *x* in **X**) **return** in **Bool**

pre **true**.

post The returned value is **true** if *x* appears at one or more nodes in *t* and is **false** otherwise.

To implement *INTREE*, we can think recursively. The item *x* will appear in the tree *t* if one of three things is the case.

1. The item *x* appears at the root of *t*.
2. The item *x* appears in the left subtree of *t*.
3. The item *x* appears in the right subtree of *t*.

In 1 above, we see how to “process the root of *t*”. We should return **true** in this case, which will provide a stopping condition.

Each of 2 and 3 is naturally dealt with through a recursive call of *INTREE*.

There is one thing missing before we give the implementation. Each of the points 1, 2 and 3 gives a condition where **true** should be returned. When should we return **false**?

If *t* is empty, then *x* certainly does not appear in *t*, and we should return **false**. This gives a second stopping condition. In general, if neither stopping condition holds, *x* will appear in *t* if and only if it appears in either the left or right subtree of *t*, and we can evaluate *INTREE*(*t*) as follows.

$INTREE(t.leftTree(), x) \vee INTREE(t.rightTree(), x)$

A recursive implementation of *INTREE* is given below.

```
function INTREE(t,x)
{
    //recInTree
    var b in Bool
    if (t.isEmpty()) then
    {
        b <-- false                                //(S1)
    }
    else
    {
        if (x == t.getRoot()) then
        {
            b <-- true                                //(S2)
        }
        else
        {
            b <-- INTREE(t.leftTree(),x)\INTREE(t.rightTree(),x)    //(R)
        }
    }
    return b
}
```

Activity 3.5

Follow the execution of the implementation *recInTree* in order to evaluate $INTREE(t, 33)$ when t is the tree shown in Figure 3.4(a). Each step in your evaluation should involve a single use of either statement (S1), (S2) or (R), and you should show which of these statements is used at each step.

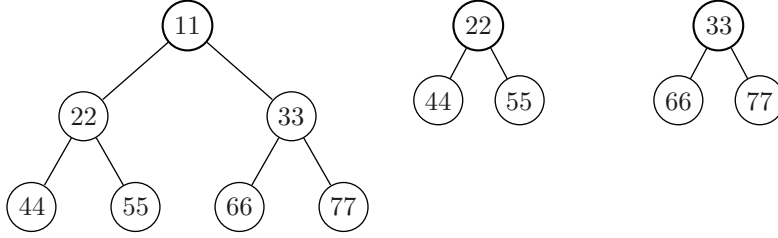


Figure 3.4 (a) The input tree t (b) t_1 (c) t_2

Discussion For the tree t in Figure 3.4(a), t is not empty, and the root of t is 11, which is not equal to 33. Hence, from (R):

$$INTREE(t, 33) = INTREE(t_1, 33) \vee INTREE(t_2, 33), \quad (1)$$

where t_1 is the tree in Figure 3.4(b), and t_2 is the tree in Figure 3.4(c).

Now t_1 is not empty, and its root is 22, which is not equal to 33. So we use (R) to get:

$$INTREE(t_1, 33) = INTREE(\textcircled{44}, 33) \vee (INTREE(\textcircled{55}, 33)). \quad (2)$$

Now the tree $\textcircled{44}$ is not empty, and its root is 44, which is not equal to 33. So we have, using (R),

$$\begin{aligned} INTREE(\textcircled{44}, 33) &= INTREE(ET()) \vee INTREE(ET()) \\ &= \mathbf{false} \vee \mathbf{false} \quad (\text{using (S1) twice}) \\ &= \mathbf{false}. \end{aligned}$$

Similarly, since the tree $\textcircled{55}$ has root 55, which is not 33, we have

$$\begin{aligned} INTREE(\textcircled{55}, 33) &= INTREE(ET()) \vee INTREE(ET()) \quad (\text{using (R)}) \\ &= \mathbf{false} \vee \mathbf{false} \quad (\text{using (S1) twice}) \\ &= \mathbf{false}. \end{aligned}$$

Then Equation (2) gives

$$INTREE(t_1, 33) = \mathbf{false} \vee \mathbf{false} = \mathbf{false}.$$

We now return to (1), and look at the term relating to the right subtree of t . Now t_2 is not empty, and its root is 33. This is equal to the input item, 33. So we use (S2) to get:

$$INTREE(t_2, 33) = \mathbf{true}.$$

Then Equation (1) gives

$$INTREE(t, 33) = \mathbf{false} \vee \mathbf{true} = \mathbf{true}.$$

When writing out an evaluation involving trees, you can use either tree diagrams or the linear notation for trees — whichever you find more convenient.

In the discussion here, we use the tree notation $\textcircled{44}$

rather than the linear notation $(44 () ())$ to show a tree consisting of just a root node 44, with empty right and left subtrees.

Suppose that we were to follow the evaluation of $INTREE(t, 100)$, where t is the tree in Figure 3.4(a), in a similar way. In that case, we would examine the item at every node in t before we could conclude that 100 does not appear in t . Searching in an unsorted binary tree is no more efficient than searching in an unsorted vector — in either case, one may need to examine every member of the collection to determine whether a particular item is in it.

However the situation is very different for a BST.

- We again first compare the item at the root with the target item x being searched for. If these are the same, then we can return **true** and stop searching.
- If the two items are different then we do not in this case need to process both of the subtrees. In a BST we know that smaller elements lie in the left subtree and larger elements lie in the right subtree. Consequently, we only have to process one of these subtrees using a recursive call.

A function $INBST$, and a recursive implementation of it, corresponding to searching in a BST, are given below.

function $INBST(t$ in **BinTree** of **X**, x in **X**) **return** in **Bool**

pre t is a BST.

post The returned value is **true** if x appears at one or more nodes in t and is **false** otherwise.

```
function INBST(t,x)
{
    //recInBST
    var b in Bool
    if (t.isEmpty()) then
    {
        b <-- false                //(S1)
    }
    else
    {
        if (t.getRoot() == x) then
        {
            b <-- true              //(S2)
        }
        else
        {
            if (x < t.getRoot()) then
            {
                b <-- INBST(t.leftTree(),x)    //(R1)
            }
            else
            {
                b <-- INBST(t.rightTree(),x)    //(R2)
            }
        }
    }
}
return b
}
```

Activity 3.6

Follow the execution of the implementation *recInBST* in order to evaluate *INBST*(*t*, 33) when *t* is the tree shown in Figure 3.5. Each step in your evaluation should involve a single use of either statement (S1), (S2), (R1) or (R2), and you should show which of these statements is used at each step.

Discussion For the tree *t* in Figure 3.5:

t.isEmpty() is **false**;

the root of *t* is 11, and $11 == 33$ is **false**;

also, $33 < 11$ is **false**.

So statement (R2) is used, to give

$$INBST(t, 33) = INBST(t_2, 33), \quad (1)$$

where *t*₂ is the right subtree of *t*, as shown in Figure 3.6.

For the tree *t*₂ in Figure 3.6:

t.isEmpty() is **false**;

the root of *t*₂ is 33, and $33 == 33$ is **true**.

So statement (S2) is used, to give

$$INBST(t_2, 33) = \mathbf{true}. \quad (2)$$

Combining (1) and (2), we get

$$INBST(t, 33) = \mathbf{true}.$$

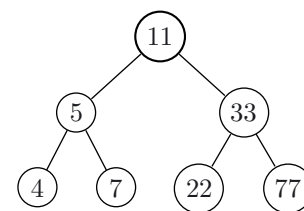


Figure 3.5

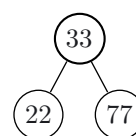


Figure 3.6

Searching in a BST is significantly quicker than searching an arbitrary binary tree. We shall investigate this in more detail in Unit 15.

Objectives for Section 3

After studying this section you should be able to do the following.

- ☐ Give a recursive implementation for a suitable function with a binary tree input.
- ☐ Given a recursive implementation involving binary trees, trace the evaluation for a particular input, showing which statement is used at each step in the evaluation.
- ☐ Determine the pre-order traversal of a given binary tree.
- ☐ Give a recursive implementation that takes account of the structure of a BST.

Exercises on Section 3

Exercise 3.1

In Example 3.1, we gave the specification of a function *ADDONET*, and an implementation *recAddOneT* of that function.

(a) By following the execution of the implementation *recAddOneT*, evaluate *ADDONET*(*t*) when *t* is the tree shown in Figure 3.7.

(b) Using recursion, give a formal postcondition for the function *ADDONET*.

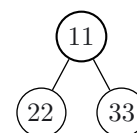


Figure 3.7

Exercise 3.2

Give a recursive implementation for the function *DOUBLE*, specified below.

function *DOUBLE*(*t* in **BinTree** of **X**) **return** in **BinTree** of **X**

pre **true**.

post The returned value is the tree formed by multiplying the integer at each node in *t* by 2.

Exercise 3.3

Give a recursive implementation for the function *BIGGEST*, specified below. Give an “efficient” implementation, which exploits the fact that the input tree is a BST. What changes would be needed to the implementation if the input could be a general tree (not necessarily a BST)?

function *BIGGEST*(*t* in **BinTree** of **Int**) **return** in **Int**

pre *t* is not empty and *t* is a BST.

post The returned value is the largest integer in the tree *t*.

4 Recursion and Games

Recursive implementations are often used in programs for games and puzzles, such as “noughts and crosses”, draughts, chess, magic squares and the like. A recursive implementation calls itself (repeatedly) to evaluate the value to be returned. Eventually it will reach the stopping condition and the recursion will stop. However, before the stopping condition is reached, information about each of the calls made will have been pushed onto a stack held in memory. (A stack is a suitable type to hold the pending information, since the calls will be “stacked up” in the correct order for retrieval as each of the results become available.) This makes recursion ideal for certain games. With the recursive calls kept on a stack, it is relatively simple to “backtrack” and replay moves.

We will look at one particular puzzle called the “Towers of Hanoi”. This puzzle was invented by the French mathematician E. Lucas, appearing in 1883 under the name M. Claus (the surname being an anagram of Lucas).

Simply put, the puzzle involves three pegs, and a number of discs, which are all of different sizes. (The original puzzle had six discs.) The discs are initially stacked on one of the pegs, with the smallest being on top of the second smallest and so on, so that disc size increases as we go down (see Figure 4.1(a)). The objective is to move the entire tower of discs to one of the other pegs, by moving one disc at a time to either of the other two pegs, but never placing a larger disc on top of a smaller one.

Can we solve this problem? And if we can, how many moves are needed?

This puzzle lends itself to a recursive approach to the solution. Suppose we want to move *n* discs from peg 1 to peg 2. A simpler problem would be to move *n* − 1 discs.

Read Only

The material in this section is not assessed directly.

See Dewdney, A. K., (1993) *The (New) Turing Omnibus* Computer Science Press, New York, p. 363.

The puzzle is also discussed at <http://www.cut-the-knot.org/recurrence/hanoi.shtml>.

If we can solve this reduced problem, then we will be able to solve the original problem. First, move the top $n - 1$ disks from peg 1 to peg 3 (Figure 4.1(b)). Then move the largest disc from peg 1 to peg 2 (Figure 4.1(c)). Finally move the $n - 1$ discs back from peg 3 to peg 2 (Figure 4.1(d)), using our strategy for the reduced problem.

This recursive view enables us to reduce the size of the problem to be solved by 1. We can do this repeatedly, until we finally need to solve a problem with only one disc. That is easy enough. If there was only one disc, we just move it from peg 1 to peg 2.

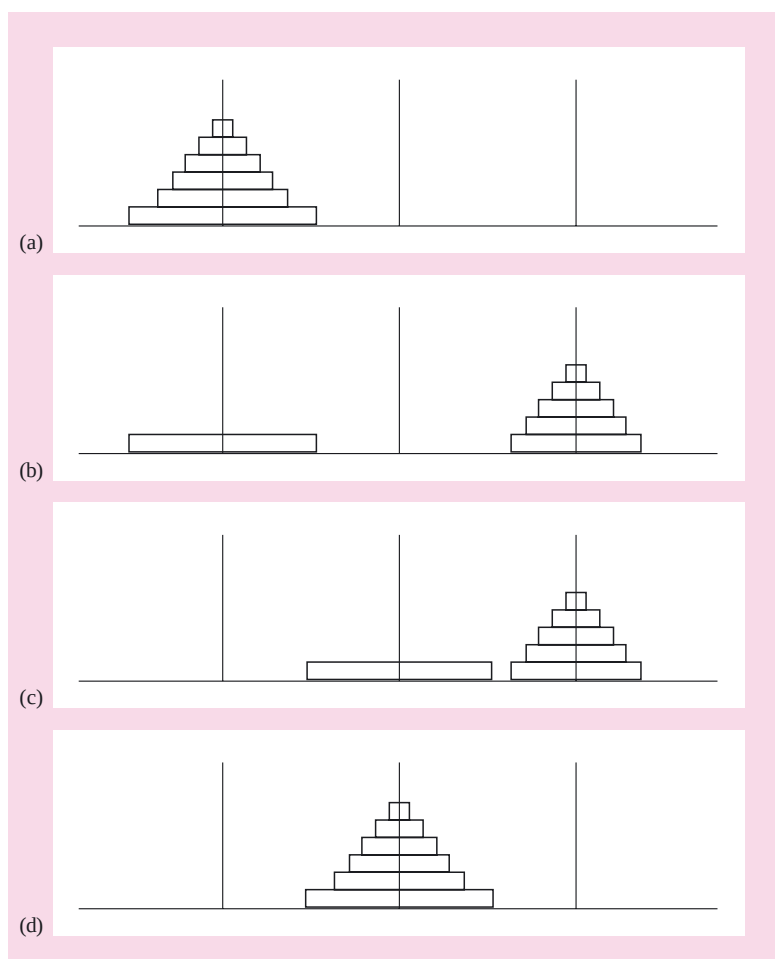


Figure 4.1 The puzzle of the “Towers of Hanoi”.

This strategy shows that the problem can be solved, but does not make it easy to solve the problem by hand — we need to remember and interpret all the “pending” moves required by the recursion. However, in principle, a machine can store this pending information. (One cautionary note: such a recursive implementation is a heavy user of memory space, so we may run out of memory if we try to solve the problem with too many discs.)

With this recursive method of solution, we can see how many moves are required — as a recurrence system. Let $HANOI(n)$ be the number of moves required by our solution if the number of discs is n . If there is just one disc (n is 1), then just one move is needed (move the single disc from peg 1 to peg 2). So $HANOI(1) = 1$. If $n > 1$, then we require:

$HANOI(n - 1)$ moves to get $n - 1$ discs from peg 1 to peg 3;

1 move to get the largest disc from peg 1 to peg 2;

then $HANOI(n - 1)$ moves to get $n - 1$ discs from peg 3 to peg 2.

This gives the recurrence system below for the function *HANOI*.

$$\begin{aligned} HANOI(1) &= 1 & (S) \\ HANOI(n) &= 2 \times HANOI(n-1) + 1 \quad (n > 1) & (R) \end{aligned}$$

A recursive implementation for *HANOI* is given below.

```
function HANOI(n)
{
  if (n == 1) then
  {
    return 1
  }
  else
  {
    return 2*HANOI(n-1) + 1
  }
}
```

EXAMPLE 4.1

We can use the recurrence system to calculate the first few values of *HANOI*, as follows.

$HANOI(1) = 1$, from (S).

Then, using (R):

$HANOI(2) = 2 \times 1 + 1 = 3$,

$HANOI(3) = 2 \times 3 + 1 = 7$,

$HANOI(4) = 2 \times 7 + 1 = 15$,

$HANOI(5) = 2 \times 15 + 1 = 31$,

$HANOI(6) = 2 \times 31 + 1 = 63$.

As we increase the number of discs, the number of moves required very quickly becomes large. Some of the more fanciful presentations of the problem have priests moving 64 golden discs from one peg to another. The time taken for this task, even moving discs at the rate of one per second, is astronomical: it would take the priests more time to complete the task than the best estimates for the age of the universe.

We shall return to this recurrence system in Unit 14. There, we shall give a formula for $HANOI(n)$ for any integer $n > 0$, and we will be able to prove that the formula gives the same values as does the recurrence system.

To implement a full solution, showing how to move the discs, is more difficult. We need to represent the problem itself. We shall not give code here to implement the full solution.

Overleaf, we give a representation of the moves needed to solve the problem for four discs. The discs on each peg are shown as a sequence of integers, where 4 represents the largest disc, down to 1 for the smallest disc. The top of the peg is the right-hand end of the sequence. We see that it takes 15 moves to transfer the four discs.

Suitable code will be supplied as part of the Practical Computing activities for this unit.

We saw in Example 4.1 that $HANOI(4) = 15$.

Moves for four discs

peg 1	peg2	peg3	
[4, 3, 2, 1]	[]	[]	The starting state.
[4, 3, 2]	[]	[1]	
[4, 3]	[2]	[1]	
[4, 3]	[2, 1]	[]	
[4]	[2, 1]	[3]	
[4, 1]	[2]	[3]	
[4, 1]	[]	[3, 2]	
[4]	[]	[3, 2, 1]	This completes the move for 3 discs from peg 1 to peg 3.
[]	[4]	[3, 2, 1]	Move the bottom disc from peg 1 to peg 2.
[]	[4, 1]	[3, 2]	Now move the three discs from peg 3 to peg 2.
[2]	[4, 1]	[3]	
[2, 1]	[4]	[3]	
[2, 1]	[4, 3]	[]	
[2]	[4, 3]	[1]	
[]	[4, 3, 2]	[1]	
[]	[4, 3, 2, 1]	[]	

Finally, we should point out that in 1980 mathematicians discovered a different, iterative solution to the problem. Both solutions take the same number of moves, which can be shown to be the minimum number of moves needed. However, the iterative solution makes far smaller demands on computer memory required than does the recursive solution.

Buneman, P. and Levy, L. (1980) "The towers of Hanoi problem" *Information Processing Letters* vol 10(4, 5) pp. 243–244.

Summary of Unit 13

1. A recurrence system provides a recursive description of a function with signature **Int** → **Int**. Recursion can be used either to provide an implementation of the function, or to express the postcondition in a specification formally.
2. Recursion can be used for functions with inputs of various types, such as stacks or strings. For valid recursion, we require: a stopping condition; a recursive call, and termination.
3. Recursion is particularly useful for functions on binary trees.

Solutions to the exercises

Solution 1.1

(a) A recursive implementation for T is given below. The stopping condition is ($n == 1$), and is the **if** condition in statement (C).

```
function T(n)
{
    //recT
    if (n == 1) then          //(C)
    {
        return 2              //(S)
    }
    else
    {
        return 2*T(n-1) + 3    //(R)
    }
}
```

(b) The step-by-step evaluation of $T(4)$ is given below.

$$\begin{aligned}
 T(4) &= 2 \times T(3) + 3 && \text{(by R)} \\
 &= 2 \times (2 \times T(2) + 3) + 3 && \text{(by R)} \\
 &= 2 \times (2 \times (2 \times T(1) + 3) + 3) + 3 && \text{(by R)} \\
 &= 2 \times (2 \times (2 \times 2 + 3) + 3) + 3 && \text{(by S)} \\
 &= 2 \times (2 \times 7 + 3) + 3 = 2 \times 17 + 3 = 37.
 \end{aligned}$$

Solution 1.2

(a) The stopping condition occurs for an input integer 0, when 1 is returned. If $n > 0$ then $POWEROFFIVE(n)$ is 5 times $POWEROFFIVE(n - 1)$. So we have the following recurrence system.

$$\begin{aligned}
 POWEROFFIVE(0) &= 1 \\
 POWEROFFIVE(n) &= 5 \times POWEROFFIVE(n - 1) \quad (n > 0)
 \end{aligned}$$

(b) We can use the recurrence system in part (a) to give a formal specification as below.

function $POWEROFFIVE(n \text{ in Int})$ **return in Int**

pre $n \geq 0$.

post $POWEROFFIVE(0) == 1$.

$(n > 0) \Rightarrow (POWEROFFIVE(n) == 5 \times POWEROFFIVE(n - 1))$.

(c) A recursive implementation is given below.

```
function POWEROFFIVE(n)
{
    if (n == 0) then
    {
        return 1
    }
    else
    {
        return 5*POWEROFFIVE(n-1)
    }
}
```

Solution 1.3

Below, we give a table tracing the execution of the given implementation when the input n is 6.

Comment	k	f	$fTwoBack$	$fOneBack$
Before the loop	-	-	1	1
	2	2	1	2
	3	3	2	3
	4	5	3	5
Loop ends	5	8	5	8

The value returned as $FIB(6)$ is 8.

Solution 2.1

Everything looks in place for a recursive implementation. There is a stopping condition and a recursive call. However, while an odd integer input will reach the stopping condition, an even integer input will not. It is essential that *all* valid inputs will eventually reach the stopping condition. For example:

$$F(5) = 1 + F(3) = 1 + (1 + F(1)) = 1 + (1 + 1) = 3;$$

$$F(4) = 1 + F(2) = 1 + (1 + F(0)) = 1 + (1 + (1 + F(-2))) = \dots$$

The attempt to find $F(4)$ will never stop.

Solution 2.2

If the input vector v is empty, then there is no 0 in v and an empty vector must be returned. This provides a stopping condition.

If v is not empty, then check the first element of v . If this is 0 then return v . This provides another stopping condition.

If the first element of v is not 0, then we use a recursive call of *CLEARTO* to process all of v except the first item. So: declare a local variable *temp*; set this to a copy of v ; remove the first item (change the state); then apply *CLEARTO* to *temp*.

An implementation of *CLEARTO* is given below.

```
function CLEARTO(v)
{
  var temp in Vector of Int
  temp <-- v.clone()
  if (v.size() == 0) then
  {
    return temp
  }
  else
  {
    if (v.at(1) == 0) then
    {
      return temp
    }
    else
    {
      temp.removeFirst()      // change state
      return CLEARTO(temp)
    }
  }
}
```

Solution 2.3

We evaluate $F(s, t)$ when s is $[5, 3, 1]$ and t is $[4, 2]$.

$$\begin{aligned}
 F([5, 3, 1], [4, 2]) &= \text{PUSH}(1, \text{PUSH}(2, F([5, 3], [4]))) && \text{(by R)} \\
 &= \text{PUSH}(1, \text{PUSH}(2, \text{PUSH}(3, \text{PUSH}(4, F([5], [])))) && \text{(by R)} \\
 &= \text{PUSH}(1, \text{PUSH}(2, \text{PUSH}(3, \text{PUSH}(4, [5])))) && \text{(by S2)} \\
 &= \text{PUSH}(1, \text{PUSH}(2, \text{PUSH}(3, [5, 4]))) \\
 &= \text{PUSH}(1, \text{PUSH}(2, [5, 4, 3])) \\
 &= \text{PUSH}(1, [5, 4, 3, 2]) \\
 &= [5, 4, 3, 2, 1].
 \end{aligned}$$

Solution 2.4

$$\begin{aligned}
 (a) \text{ CODE}(\text{"abC"}) &= \text{CODE}(\text{"ab"}).\text{charAppend}(\text{CHR}(\text{ASC}(\text{'C'}) + 1)) && \text{(by R)} \\
 &= \text{CODE}(\text{"ab"}).\text{charAppend}(\text{'D'}) \\
 &= (\text{CODE}(\text{"a"}).\text{charAppend}(\text{'c'})).\text{charAppend}(\text{'D'}) && \text{(by R)} \\
 &= ((\text{CODE}(\text{""}).\text{charAppend}(\text{'b'})).\text{charAppend}(\text{'c'})).\text{charAppend}(\text{'D'}) && \text{(by R)} \\
 &= ((\text{""}).\text{charAppend}(\text{'b'})).\text{charAppend}(\text{'c'}).\text{charAppend}(\text{'D'}) && \text{(by S)} \\
 &= (\text{"b"}).\text{charAppend}(\text{'c'}).\text{charAppend}(\text{'D'}) \\
 &= \text{"bc"}.\text{charAppend}(\text{'D'}) \\
 &= \text{"bcD"}.
 \end{aligned}$$

(b) A suitable specification is given below.

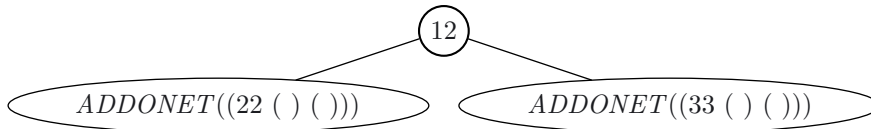
function $\text{CODE}(s \text{ in mString})$ **return in mString**

pre The string s does not contain any character with ASCII code 126.

post The returned value is the string formed by changing every character in s to the next character in the ASCII table.

Solution 3.1

(a) For the tree t in Figure 3.7, t is not empty. The root of t is 11, and $11+1 = 12$. Hence, from (R1), (R2) and (R3), $\text{ADDONET}(t)$ is the tree below (where the tree $\textcircled{22}$ is written in linear notation, as $(22 \ (\) \ (\))$, and $\textcircled{33}$ is written similarly).

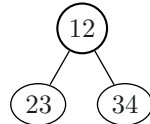


Now the tree $(22 \ (\) \ (\))$ is not empty, and has root 22. Then

$$\begin{aligned}
 \text{ADDONET}((22 \ (\) \ (\))) &= (22 + 1 \ \text{ADDONET}(\text{ET}()) \ \text{ADDONET}(\text{ET}())), && \text{by (R1), (R2) and (R3)} \\
 &= (23 \ (\) \ (\)), && \text{by the stopping condition.}
 \end{aligned}$$

Similarly, $\text{ADDONET}((33 \ (\) \ (\))) = (34 \ (\) \ (\))$.

So $\text{ADDONET}(t)$ is the tree



(b) We can give a formal postcondition as follows. Notice how this postcondition gives enough information about the returned value to enable us to recognise whether or not a proposed returned value is as required. It does not tell us how to “construct” the returned value – but that is not necessary.

$$\begin{aligned}
 \text{post} \quad & (t.\text{isEmpty}() \Rightarrow (\text{ADDONET}(t).\text{isEmpty}())) \\
 & \neg(t.\text{isEmpty}()) \Rightarrow \\
 & ((\text{ADDONET}(t).\text{getRoot}() == t.\text{getRoot}() + 1) \wedge \\
 & \quad (\text{ADDONET}(t).\text{leftTree}() == \text{ADDONET}(t.\text{leftTree}())) \wedge \\
 & \quad (\text{ADDONET}(t).\text{rightTree}() == \text{ADDONET}(t.\text{rightTree}()))).
 \end{aligned}$$

It is not necessary to lay out the postcondition like this. It is done to show the structure of the proposition.

Solution 3.2

We need to multiply the element at the root node by 2. Then set this “processed” value as the root of a tree that has had both its left subtree and its right subtree processed by *DOUBLE*.

The stopping condition is for the empty tree, the return value for this input being the empty tree. We can give a recursive implementation in a form very like that in Example 3.1.

```
function DOUBLE(t)
{
    // recDouble
    var temp in BinTree
    temp <-- t.clone()
    if (NOT(t.isEmpty())) then
    {
        temp.setRoot(t.getRoot()*2)           //(R1)
        temp.setLeftTree(DOUBLE(t.leftTree())) //(R2)
        temp.setRightTree(DOUBLE(t.rightTree())) //(R3)
    }
    return temp
}
```

Solution 3.3

The biggest number in a BST is the biggest number in its right subtree, unless this subtree is empty. If the right subtree is empty, the biggest item is at the root. This situation provides a stopping condition. A recursive implementation is given below.

```
function BIGGEST(t)
{
    if (t.rightTree().isEmpty()) then
    { return t.getRoot() }
    else
    { return BIGGEST(t.rightTree()) }
}
```

If t is not a BST we would need to use recursive calls of *BIGGEST* on both the left and right subtrees, and to write code to return the largest of the three values:

$t.getRoot()$; $BIGGEST(t.leftTree())$; $BIGGEST(t.rightTree())$.

Index

factorial numbers, 6
Fibonacci numbers, 13

recurrence system, 7
 initial condition, 11
 solution of, 11
 starting condition, 11
 stopping condition, 11

recursive call, 7

stopping condition, 7
stopping value, 7

tree traversal
 post-order, 35
 pre-order, 35
triangular numbers, 12

valid recursion, 17
 recursive call, 17
 stopping condition, 17
 termination, 17